

Sparsity-basierte gemischt-ganzzahlige Optimierung zur Identifikation algebraischer Gleichungen aus Daten

Masterarbeit von Jonathan Foltz
Tag der Einreichung: 10. August 2026

1. Gutachten: Prof. Dr. Marc Pfetsch
Darmstadt

Wirtschaftsmathematik
Optimierung

Erklärung zur Abschlussarbeit gemäß § 22 Abs. 7 APB TU Darmstadt

Hiermit erkläre ich, Jonathan Foltz, dass ich die vorliegende Arbeit gemäß § 22 Abs. 7 APB der TU Darmstadt selbstständig, ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt habe. Ich habe mit Ausnahme der zitierten Literatur und anderer in der Arbeit genannter Quellen keine fremden Hilfsmittel benutzt. Die von mir bei der Anfertigung dieser wissenschaftlichen Arbeit wörtlich oder inhaltlich benutzte Literatur und alle anderen Quellen habe ich im Text deutlich gekennzeichnet und gesondert aufgeführt. Dies gilt auch für Quellen oder Hilfsmittel aus dem Internet.

Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Mir ist bekannt, dass im Falle eines Plagiats (§ 38 Abs. 2 APB) ein Täuschungsversuch vorliegt, der dazu führt, dass die Arbeit mit 5,0 bewertet und damit ein Prüfungsversuch verbraucht wird. Abschlussarbeiten dürfen nur einmal wiederholt werden.

Bei einer Thesis des Fachbereichs Architektur entspricht die eingereichte elektronische Fassung dem vorgestellten Modell und den vorgelegten Plänen.

Darmstadt, 10. August 2026



Jonathan Foltz



Danksagung

Mein besonderer Dank gilt meinem Betreuer, Herrn Prof. Pfetsch, für die fachliche Begleitung meiner Masterarbeit. Insbesondere danke ich ihm für sein konstruktives Feedback, seine wertvollen Anregungen und die hilfreichen Hinweise, die wesentlich zur Weiterentwicklung und Fertigstellung dieser Arbeit beigetragen haben.



Einsatz von KI-Hilfsmitteln

Bei der Erstellung dieser Arbeit wurden generative KI-Hilfsmittel unterstützend eingesetzt. Die Nutzung beschränkte sich auf Hilfestellungen bei der $\text{\LaTeX}$ -Syntax, die sprachliche Korrektur selbst verfasster Textpassagen sowie die Unterstützung bei der Implementierung einiger Bestandteile der numerischen Experimente.

Die mathematische Konzeption, die Herleitung und Formulierung der Optimierungsmodelle, die Auswahl und Ausgestaltung der numerischen Experimente sowie die Auswertung und Interpretation der Ergebnisse wurden eigenständig vorgenommen. Sämtliche durch KI-Hilfsmittel erzeugten Vorschläge wurden vor ihrer Verwendung inhaltlich und technisch überprüft, angepasst und in die eigene Ausarbeitung beziehungsweise Implementierung integriert. Insbesondere wurden KI-gestützt erstellte oder überarbeitete Programmteile eigenständig getestet und auf ihre Übereinstimmung mit den beschriebenen Algorithmen und Versuchsaufbauten geprüft.

Für den Inhalt der Arbeit sowie für die Richtigkeit der dargestellten Ergebnisse trägt ausschließlich der Autor die Verantwortung. Als KI-Hilfsmittel wurde ChatGPT von OpenAI verwendet.

Zusammenfassung

Diese Arbeit untersucht die Rekonstruktion interpretierbarer algebraischer Gleichungen aus endlichen, möglicherweise verrauschten Stichproben. Hierzu wird ein sparsity-basierter Optimierungsansatz entwickelt. Dieser bestimmt Polynome, die auf den Stichproben exakt oder nahezu verschwinden. Hierfür wird die Anzahl der verwendeten Monome durch eine ℓ_0 -Bedingung in der Zielfunktion minimiert. Diese wird mithilfe von Binärvariablen und einer Big-M-Bedingung als gemischt-ganzzahliges Problem mit kontinuierlichen und ganzzahligen Koeffizienten modelliert. Ausgehend vom Spezialfall der Hyperflächen wird der Ansatz auf das Problem der symbolischen Regression und mit der Pivot- und der No-Good-Cut-Methode durch zwei sequentielle Verfahren auf allgemeine Varietäten verallgemeinert. Mithilfe von linearer Algebra und Komplexitätstheorie wird das Optimierungsproblem analysiert, durch Presolving-Maßnahmen vereinfacht und das zugehörige Entscheidungsproblem als NP-schwer klassifiziert. Numerische Experimente mit Hyperflächen, allgemeinen Varietäten und dem Feynman-Datensatz zeigen, dass die zugrunde liegenden Gleichungen insbesondere bei rauschfreien Daten und geringem bis mittleren Rauschen zuverlässig rekonstruiert werden können. Modelle mit ganzzahligen Koeffizienten erweisen sich als besonders robust, sofern eine ganzzahlige Darstellung der gesuchten Gleichungen plausibel ist. Für $SO(3)$ wurden bei rauschfreien Daten alle linear unabhängigen Gleichungen in den betrachteten Polynomräumen von der Pivot-Methode rekonstruiert. Von den 53 mit der Kandidatenbibliothek darstellbaren Feynman-Gleichungen konnten 52 rekonstruiert werden, davon 37 auch bei gaußschem Rauschen mit $\sigma = 0.1$. Im Fall der symbolischen Regression haben sich die Presolving-Maßnahmen als besonders geeignet herausgestellt. Diese konnten die Lösungszeit für die Rekonstruktion ohne Rauschen um 79% reduzieren. Im Fall der Varietäten konnte die kernbasierte Monomselektion in einigen Beispielen den Rechenaufwand deutlich reduzieren. Der Nutzen hängt jedoch von der Problemstruktur ab.

Inhaltsverzeichnis

1	Einleitung	8
2	Grundlagen aus der algebraischen Geometrie	11
3	Herleitung des Optimierungsproblems	14
3.1	Herleitung des Optimierungsproblems - Spezialfall Hyperflächen	15
3.2	Herleitung des Optimierungsproblems für allgemeine Varietäten	21
3.2.1	Optimierungsproblem für allgemeine Varietäten - Pivot-Methode	22
3.2.2	Optimierungsproblem für allgemeine Varietäten - No-Good-Cut-Methode	24
4	Komplexität des Optimierungsproblems	27
4.1	Struktur des Lösungsraums	27
4.2	Wahl der Pivotelemente	33
4.3	Beweis der NP-Schwere des Problems	35
4.4	Einflussfaktoren auf das Branch-and-Bound Verhalten	38
4.4.1	Einflussfaktor Big-M	38
4.4.2	Einflussfaktor Monomanzahl	39
4.4.3	Einflussfaktor Rang der Evaluationsmatrix	39
4.4.4	Branching ignoriert Korrelation von Monomen	40
4.4.5	Einflussfaktor Rauschen	40
4.5	Komplexitätsreduzierung durch Presolving	41
4.5.1	ℓ_1 -Lösung als Warm-Start nutzen	41
4.5.2	Kernbasierte Monomselektion	42
4.5.3	Verfeinerung der Big-M-Bedingung	43
4.5.4	Algorithmus inklusive Presolving	45
5	Erweiterung des Modells auf symbolische Regression	48
5.1	Konstruktion der Kandidatenbibliothek	50



5.2	Komplexität der Erweiterung	51
5.2.1	Presolving-Maßnahmen	52
6	Numerische Experimente	55
6.1	Hyperflächen	55
6.2	Allgemeine Varietäten	65
6.3	Symbolische Regression	70
7	Fazit	77

1 Einleitung

In vielen naturwissenschaftlichen und technischen Anwendungen liegen Daten vor, von denen angenommen wird, dass sie einer unbekannten mathematischen Struktur folgen. Ein Ziel der Datenanalyse besteht darin, diese Struktur anhand endlich vieler Messdaten zu rekonstruieren. Dabei ist nicht nur eine möglichst genaue Beschreibung der Daten von Interesse. Häufig soll zusätzlich ein Modell bestimmt werden, das einfach interpretierbar ist und Rückschlüsse auf die zugrunde liegenden Zusammenhänge erlaubt. [4]

Die Ausgangssituation ist also durch eine endliche Menge von Stichprobendaten

$$X = \{x^{(1)}, \dots, x^{(N)}\} \subseteq \mathbb{R}^d$$

gegeben. Wir nehmen in dieser Arbeit an, dass diese Stichprobendaten Messwerte einer algebraischen Gleichung sind. Zunächst betrachten wir dabei Stichproben, die auf oder in der Nähe einer unbekannten algebraischen Varietät liegen. Das Ziel besteht hier darin, anhand der Stichproben Polynomgleichungen zu bestimmen, deren gemeinsame Nullstellenmenge die zugrunde liegende Varietät möglichst gut beschreibt. Im einfachsten Fall handelt es sich bei der Varietät um eine Hyperfläche, die durch eine einzelne Gleichung beschrieben werden kann. Allgemeine Varietäten benötigen dagegen möglicherweise mehrere Polynomgleichungen.

Die theoretische Motivation für diesen Ansatz liefert die algebraische Geometrie [25]. Einer algebraischen Varietät können wir ihr Verschwindungsideal zuordnen. Dieses besteht aus allen Polynomen, die auf der Varietät verschwinden. Der Hilbertsche Basissatz garantiert eine endliche Beschreibung algebraischer Varietäten. Daraus leiten wir eine Motivation her, um anhand von Daten die Polynome zu bestimmen, deren gemeinsame Nullstellenmengen eine Varietät beschreiben. Wir greifen das Grundproblem aus [4] auf, das von den Autoren durch Singulärwert- bzw. QR-Zerlegung angegangen wurde. Die Autoren von [12] nutzen einen Approximate-Vanishing-Ideals Algorithmus, um Polynome zu bestimmen, die auf den Stichproben nahezu verschwinden und eine approximative Borderbasis bilden. Im Gegensatz zu diesen Ansätzen liegt der Schwerpunkt dieser Arbeit

nicht nur auf der vollständigen Berechnung einer Basis des Verschwindungsideals. Da auf einer endlichen Punktmenge mehrere verschiedene Polynomgleichungen verschwinden können, ist unser Ziel eine möglichst einfache Repräsentation der Polynomgleichungen zu erzielen. Dieses Ziel verfolgen wir durch die Formulierung des Problems als gemischt-ganzzahliges lineares Programm (MILP) mit einer ℓ_0 -sparsity Bedingung. Diese Bedingung formulieren wir in dieser Arbeit mithilfe von Binärvariablen und einer Big-M-Bedingung. Garey und Johnson haben in [14] die NP-Schwere eines sparsen Approximationsansatzes mit ℓ_0 -Norm bewiesen. Natarajan hat dieses Resultat in seiner Arbeit [21] auf approximative sparse Probleme verallgemeinert. Aufbauend auf diesen NP-Schwere-Resultaten wurden weitere sparsity-Probleme als NP-schwer klassifiziert [22]. Auch wir nutzen Gareys und Johnsons Resultat in dieser Arbeit, um durch eine Reduktion des Problems von Garey und Johnson auf eine Entscheidungsvariante unseres Problems die NP-Schwere unserer Entscheidungsvariante zu zeigen. Aufbauend auf diesem Resultat besprechen wir Faktoren, die die Komplexität für einen Branch-and-Bound-Solver beeinflussen. Um kritische Faktoren zu reduzieren, betrachten wir drei verschiedene Presolving-Maßnahmen zur Komplexitätsreduzierung.

Abschließend übertragen wir den erarbeiteten Ansatz auf ein Grundproblem der symbolischen Regression. Dabei wird anhand von Eingabe-Ausgabe-Paaren eine explizite mathematische Gleichung rekonstruiert. Eine frühe Version der symbolischen Regression geht auf die genetische Programmierung zurück [17]. Bei dieser werden Funktionsbausteine automatisch und dynamisch während der Optimierung erzeugt. Später wurden diese Gedanken von Schmidt und Lipson auf die automatische Entdeckung von Naturgesetzen aus experimentellen Daten übertragen [29]. Einen modernen Ansatz liefern Udrescu und Tegmark in [34]. Das dort entwickelte Verfahren kombiniert die Approximation einer unbekannten physikalischen Gleichung durch ein neuronales Netz mit Dimensionsanalyse, Symmetrierkennung und Separierbarkeitstests. Im Gegensatz zu diesen Ansätzen erzeugen wir eine endliche Kandidatenbibliothek aus möglichen Funktionsbausteinen und bestimmen anschließend mithilfe einer angepassten Version des MILPs für Varietäten eine sparse Linearkombination dieser Kandidaten. Das dabei entstehende MILP vereinfachen wir erneut durch Presolving-Maßnahmen.

Sowohl das MILP für Varietäten als auch das für symbolische Regression, testen wir zum Schluss in numerischen Experimenten. Wir betrachten zunächst mit dem Einheitskreis, der Lemniskate von Geronio und den Schnitten des Einheitskreises mit einer Geraden verschiedene Hyperflächen. An diesen Beispielen testen wir in erster Linie die generelle Erfolgsquote unter Rauschen mit ganzzahligen und kontinuierlichen Koeffizienten, die entwickelten Presolving-Maßnahmen und die minimale Anzahl benötigter Daten. Daraufhin betrachten wir zwei allgemeine Varietäten mit $SO(3)$ und den 2×3 -Matrizen von

Rang 1. Wir betrachten vor allem die Lösbarkeit unter verrauschten Daten. Zum Schluss betrachten wir das Problem symbolischer Regression, indem wir unseren Algorithmus an dem Feynman-Datensatz aus [34] testen.

Die Hauptbeiträge dieser Arbeit lassen sich dabei wie folgt zusammenfassen:

1. Formulierung und Vergleich von sparsity-basierten Optimierungsproblemen in einer ℓ_0 -Pseudonorm Variante und einer ℓ_1 -Norm Variante mit ganzzahligen bzw. kontinuierlichen Koeffizienten zur Bestimmung von interpretierbaren Polynomgleichungen zur Beschreibung von Hyperflächen aus Daten.
2. Verallgemeinerung auf allgemeine Varietäten durch Herleitung eines sequentiellen Ansatzes und Verallgemeinerung auf symbolische Regression.
3. Komplexitätsanalyse inklusive Klassifizierung des zugehörigen Entscheidungsproblems als NP-schweres Problem durch Reduktionsbeweis, Analyse des Branch-and-Bound-Solververhaltens und Komplexitätsreduzierung durch verschiedene Presolvingmaßnahmen.
4. Numerische Experimente mit verschiedenen Hyperflächen, Varietäten und Feynman-Gleichungen zur Untersuchung der theoretisch getroffenen Aussagen.

Die Arbeit ist wie folgt aufgebaut. Zunächst führen wir in Kapitel 2 die benötigten Grundlagen aus der algebraischen Geometrie ein. Daraufhin leiten wir in Kapitel 3 das Optimierungsproblem für Hyperflächen her und erweitern es auf allgemeine Varietäten. Anschließend untersuchen wir in Kapitel 4 die Komplexität des Problems, entwickeln verschiedene Presolving Strategien und formulieren einen Algorithmus. Im Anschluss übertragen wir in Kapitel 5 den Ansatz auf symbolische Regression, diskutieren die Komplexitätsveränderung durch die Erweiterung und leiten Presolving-Maßnahmen her. Abschließend testen wir in Kapitel 6 die entstehenden Algorithmen in numerischen Experimenten.

2 Grundlagen aus der algebraischen Geometrie

In diesem Kapitel führen wir die grundlegenden Begriffe ein, die wir für den weiteren Verlauf dieser Arbeit zur Beschreibung algebraischer Varietäten benötigen. Wir beschränken uns in diesem Kapitel auf Grundbegriffe, die für das Verständnis dieser Arbeit nötig sind. Für einen tiefergehenden Einstieg in das Thema sei auf [25] verwiesen. An diesem Werk orientieren sich auch Aufbau und Inhalt dieses Kapitels.

Sei $\mathbb{R}[x_1, \dots, x_n]$ der Polynomring in n Variablen über $\mathbb{R}$. Ist $T \subset \mathbb{R}[x_1, \dots, x_n]$ eine Menge von Polynomen, dann schreiben wir

$$\mathcal{V}(T) = \{p \in \mathbb{R}^n \mid h(p) = 0 \text{ für alle } h \in T\}$$

für die Nullstellenmenge der Polynome in T . Wir wiederholen nun zunächst die Definition eines Ideals.

Definition 2.1. Eine nichtleere Teilmenge $I \subseteq \mathbb{R}[x_1, \dots, x_n]$ heißt Ideal, wenn

(i) $f, g \in I \Rightarrow f + g \in I$ und

(ii) $f \in I, h \in \mathbb{R}[x_1, \dots, x_n] \Rightarrow hf \in I$.

Für $T \subseteq \mathbb{R}[x_1, \dots, x_n]$ bezeichnen wir mit (T) das von T erzeugte Ideal.

Da $\mathcal{V}(T) = \mathcal{V}((T))$ gilt, genügt es, Nullstellenmengen über Ideale zu betrachten. Wir definieren nun den, für unsere Arbeit zentralen Begriff einer Varietät.

Definition 2.2. Sei $V \subseteq \mathbb{R}^n$. Wir nennen V eine affine Varietät, wenn $V = \mathcal{V}(T)$ für irgendeine Menge T von Polynomen in $\mathbb{R}[x_1, \dots, x_n]$.

In dieser Arbeit wird der Begriff Varietät für affine algebraische Mengen verwendet. Andere Autoren setzen für den Begriff zusätzlich Irreduzibilität voraus. Diese wird jedoch für den Kontext dieser Arbeit nicht benötigt. Nach der Definition werden affine Varietäten als gemeinsame Nullstellenmengen von Polynomfamilien beschrieben. Um eine endliche Beschreibung von algebraischen Varietäten zu bekommen, nutzen wir folgenden Satz.

Satz 2.3 (Hilbertscher Basissatz). *Im Polynomring $\mathbb{R}[x_1, \dots, x_n]$ ist jedes Ideal $I \subseteq \mathbb{R}[x_1, \dots, x_n]$ endlich erzeugt: Es existieren $h_1, \dots, h_r \in I$ mit*

$$I = (h_1, \dots, h_r)$$

Beweis. Siehe [25, S. 151]. □

Ist $V = \mathcal{V}(T)$ und $I = (T)$, dann existieren somit $h_1, \dots, h_r \in I$ mit

$$I = (h_1, \dots, h_r).$$

Daher gilt

$$V = \mathcal{V}(T) = \mathcal{V}(I) = \mathcal{V}(h_1, \dots, h_r)$$

Der Hilbertsche Basissatz garantiert somit eine endliche Beschreibung algebraischer Varietäten.

Definition 2.4. *Eine affine Varietät, die durch eine einzige Gleichung gegeben ist, also von der Form $\mathcal{V}(f) \subset \mathbb{R}^n$, für ein nichtkonstantes Polynom $f \in \mathbb{R}[x_1, \dots, x_n]$, heißt eine affine Hyperfläche.*

Wir definieren weiter für den späteren Verlauf dieser Arbeit das Verschwindungsideal, um Punktmengen durch die Polynome zu beschreiben, die auf ihnen verschwinden.

Definition 2.5. *Für $S \subseteq \mathbb{R}^n$ definieren wir das Ideal*

$$\mathcal{I}(S) = \{f \in \mathbb{R}[x_1, \dots, x_n] \mid f(p) = 0 \text{ für alle } p \in S\}.$$

Insbesondere ist $\mathcal{I}(V)$ für eine algebraische Menge V das Ideal aller Polynome, die auf V verschwinden.

Das Ziel dieser Arbeit ist, aus einer endlichen Stichprobenmenge $X \subseteq V$ Polynome zu bestimmen, deren gemeinsame Nullstellenmenge Informationen über die Varietät V liefert.

Bemerkung 2.6. Über algebraisch abgeschlossenen Körpern, beispielsweise $\mathbb{C}$, beschreibt der Hilbertsche Nullstellensatz die Beziehung zwischen Idealen und ihren Nullstellenmengen. Da wir in dieser Arbeit jedoch mit endlichen reellen und teilweise verrauschten Stichproben arbeiten, wird die Identifizierbarkeit nicht aus dem Nullstellensatz gerechtfertigt. Sie wird später anhand der Evaluationsmatrix, ihres Kerns und geeigneter Rangbedingungen untersucht. Für weiterführende Informationen hinsichtlich des Hilbertschen Nullstellensatz sei auf [25, S.14] verwiesen.

Die Suche im reellen Fall motiviert sich dadurch, dass für den rauschfreien Fall aus $X \subseteq V$ die Inklusion $\mathcal{I}(V) \subseteq \mathcal{I}(X)$ folgt. Die gegensätzliche Inklusion gilt jedoch im Allgemeinen nicht. Auf endlich vielen Stichproben der Varietät verschwinden also zusätzliche Polynome, die keine Gleichungen der gesamten Varietät sind. Es kann also nicht unser Anspruch sein aus einer endlichen Stichprobenmenge eine vollständige Beschreibung der Varietät zu bekommen. Deshalb beschränken wir die Suche auf Polynome bis zu einem vorgegebenen Maximalgrad und bevorzugen sparse Polynome. Die Frage, unter welchen Bedingungen eine gesuchte Gleichung anhand der Stichprobe identifiziert werden kann, wird in Kapitel 4 mithilfe der Evaluationsmatrix untersucht.

3 Herleitung des Optimierungsproblems

Ziel dieses Kapitels ist es, ein Optimierungsproblem herzuleiten. Mit diesem rekonstruieren wir aus einer endlichen Menge von Stichprobenpunkten algebraische Gleichungen, die auf den Daten nahezu verschwinden. Die Hoffnung ist, dass die Stichproben repräsentativ für die Varietät sind, wodurch die gefundenen Gleichungen auch auf der Varietät gelten.

Sei dazu eine endliche Punktmenge

$$X = \{x^{(1)}, \dots, x^{(N)}\} \subset \mathbb{R}^d, \quad d \in \mathbb{N}.$$

gegeben, die als (möglicherweise verrauschte) Stichprobe einer unbekannten algebraischen Varietät

$$V \subset \mathbb{R}^d$$

aufgefasst wird. Im Folgenden versuchen wir, möglichst viele Informationen über das Verschwindungsideal

$$\mathcal{I}(V)$$

zu bestimmen. Weiterhin betrachten wir in dieser Arbeit nicht den ganzen Raum $\mathbb{R}[x_1, \dots, x_d]$, sondern lassen nur Monome bis zu einem Maximalgrad D zu. Dazu definieren wir die Menge der Monome bis Grad D in d Variablen durch

$$\mathcal{M} = \{x_1^{\alpha_1} x_2^{\alpha_2} \cdots x_d^{\alpha_d} \mid \alpha_1 + \cdots + \alpha_d \leq D, \alpha_i \in \mathbb{N}_0, i = 1, \dots, d\}.$$

Die Menge $\mathcal{M}$ bildet eine Basis des Vektorraums

$$\mathbb{R}[x_1, \dots, x_d]_{\leq D} = \{f \in \mathbb{R}[x_1, \dots, x_d] \mid \deg(f) \leq D\}.$$

Damit lassen sich durch Linearkombinationen der Elemente von $\mathcal{M}$ alle Polynome in $\mathbb{R}[x_1, \dots, x_d]_{\leq D}$ darstellen.

Da wir mit endlichen Stichproben und den Einschränkungen bezüglich des maximalen Monomgrades nicht erwarten können eine exakte Beschreibung von $\mathcal{I}(V)$ zu lernen, ist das Ziel unseres späteren Optimierungsmodells, eine Lösung für

$$\mathcal{I}(X) \cap \mathbb{R}[x_1, \dots, x_d]_{\leq D} \quad (3.1)$$

zu finden. Die Hoffnung ist, dass

$$\mathcal{I}(X) \cap \mathbb{R}[x_1, \dots, x_d]_{\leq D} \approx \mathcal{I}(V) \cap \mathbb{R}[x_1, \dots, x_d]_{\leq D}.$$

Im Folgenden behandeln wir zunächst den Spezialfall von Hyperflächen. Wie wir im letzten Abschnitt gesehen haben, existiert in diesem Fall ein nichttriviales Polynom $f \in \mathbb{R}[x_1, \dots, x_d]$ mit

$$V = \{x \in \mathbb{R}^d \mid f(x) = 0\},$$

sodass eine einzelne Polynomgleichung zur vollständigen Beschreibung der Varietät ausreicht. Anschließend erweitern wir das Modell auf allgemeine Varietäten.

3.1 Herleitung des Optimierungsproblems - Spezialfall Hyperflächen

In diesem Kapitel nehmen wir zunächst an, dass X Stichprobenpunkte einer unbekannten Hyperfläche sind. Da $\mathcal{M}$ wie eben definiert eine Basis des Vektorraums $\mathbb{R}[x_1, \dots, x_d]_{\leq D}$ ist, lässt sich ein beliebiges Polynom $f \in \mathbb{R}[x_1, \dots, x_d]_{\leq D}$ darstellen durch

$$f(x) = \sum_{j=1}^M c_j m_j(x).$$

Unser Ziel ist es nun im Folgenden die Parameter c_j so zu bestimmen, dass wir ein nichttriviales Polynom

$$f \in \mathcal{I}(X) \cap \mathbb{R}[x_1, \dots, x_d]_{\leq D} \quad (3.2)$$

finden. Im Fall von verrauschten Daten suchen wir ein Polynom, das auf X nahezu verschwindet. Da die Daten X von einer Hyperfläche stammen, können wir annehmen, dass die Nullstellenmenge einer einzelnen Polynomgleichung zur Beschreibung der algebraischen Struktur genügt.

Im Laufe dieser Arbeit erarbeiten wir, dass die Rekonstruktion eines Polynoms im Sinne von (3.2) nicht unbedingt eindeutig ist. Aus diesem Grund ist ein weiteres Optimierungsziel, aus den verschiedenen möglichen Polynomgleichungen eine möglichst einfache Gleichung zur Beschreibung der Varietät auszuwählen. Dies versuchen wir zu erzielen, indem wir eine sparsity-Bedingung in unser Optimierungsproblem aufnehmen. Mit einer sparsity-Bedingung bevorzugen wir im Optimierungsprozess Polynome mit vielen verschwindenden Koeffizienten gegenüber Polynomen mit wenigen. Somit erhalten wir ein leicht interpretierbares Polynom [4]. Wir betrachten in dieser Arbeit zwei unterschiedliche Möglichkeiten, die in der Problemklasse gemischt-ganzzahliger linearer Probleme zu sparsity führen können. Zum einen kann eine exakte sparsity-Bedingung mithilfe der ℓ_0 -Pseudonorm

$$\|c\|_0 := \#\{j \mid c_j \neq 0\}$$

in die Zielfunktion oder in die Nebenbedingungen mit aufgenommen werden. Der Vorteil ist, dass die sparsity einfach kontrollierbar ist. Allerdings sehen wir in Abschnitt 4.3, dass das Problem mit dieser Bedingung NP-schwer ist.

Eine andere Möglichkeit wäre, die ℓ_1 -Norm

$$\|c\|_1 := \sum_{j=1}^M |c_j|$$

zu verwenden, da sie in einigen Fällen eine konvexe Approximation der ℓ_0 -Pseudonorm ist. Dieses Optimierungsproblem ist im Allgemeinen deutlich weniger komplex, da die ℓ_1 -Norm durch Einführung von Positiv- und Negativteil der Koeffizienten linear modelliert werden kann. Allerdings ist die sparsity mit der ℓ_1 -Norm nicht immer exakt, wodurch wir unter Umständen nur kleine Koeffizienten und somit keine sparsity erhalten. In der Theorie des Compressed Sensing wurden in [8] mithilfe des restricted isometry property für ähnliche Optimierungsprobleme hinreichende Bedingungen beschrieben, bei denen die Lösungen der beiden Optimierungsprobleme übereinstimmen. Die Bestimmung der besten Konstante, für die das restricted isometry property einer Matrix erfüllt ist, ist jedoch ein NP-schweres Problem [32]. Wir werden in den numerischen Experimenten sehen, dass die ℓ_0 -Variante deutlich zuverlässiger sparsity liefert. Deshalb betrachten wir als Hauptproblem zunächst das Problem mit ℓ_0 -Pseudonorm.

Unser Ziel ist also, das folgende Optimierungsproblem zu lösen.

$$\begin{aligned}
& \min \quad \|c\|_0 \\
& \text{u.d.N.} \quad \sum_{j=1}^M c_j m_j(x^{(i)}) = 0, \quad i = 1, \dots, N, \\
& \quad b^\top c = 1, \quad b \in \mathbb{R}^M \\
& \quad c_j \in \mathbb{R} \quad \quad \quad j = 1, \dots, M.
\end{aligned} \tag{3.3}$$

Die Nebenbedingung $b^\top c = 1$ mit dem Vektor $b \in \mathbb{R}^M$ ist nötig, da sonst die triviale Nulllösung stets optimal wäre. Dies liegt an der Nichtnegativität von $\|\cdot\|_0$. Für den Vektor b kann zum Beispiel $b = e_k$, $k \in \{1, \dots, M\}$ genutzt werden. Dadurch wird der k -te Lösungskoeffizient $c_k = 1$ fixiert. Zusätzlich verhindert der Vektor, dass wir Skalierungsinvarianz in der Optimallösung haben. Das bedeutet, dass unter Abwesenheit dieser Nebenbedingung für jede Optimallösung $\tilde{c}$ für $\lambda \in \mathbb{R}$ die skalierte Variante $\lambda \tilde{c}$ ebenso eine Optimallösung des Problems 3.1 ist. Diese beiden unterschiedlichen Lösungen führen zwar auf die gleiche Varietät, werden allerdings im Lösungsprozess von einem Solver als zwei individuelle Lösungen angesehen. Dies verschlechtert die Laufzeit des Solvers.

Dieses Optimierungsproblem ist allerdings nur das konzeptionelle Problem. Es ist nicht direkt mit einem gängigen Solver lösbar. Um dieses Problem lösbar zu machen, führen wir sogenannte Big-M-Bedingungen ein. Dies ist ein gängiges Verfahren, um ein Problem mit ℓ_0 -Pseudonorm als gemischt-ganzzahliges Problem zu formulieren, da es für hinreichend großes Big-M äquivalent zur Formulierung (3.3) ist [36, 37].

Problem 3.1. *Das Optimierungsproblem zur Optimierung der Parameter einer Polynomgleichung zur Beschreibung von (3.2) anhand von Stichprobenpunkten mit der ℓ_0 -Pseudonorm*

formuliert als MILP ist damit für ein $b \in \mathbb{R}^M$ gegeben durch

$$\begin{aligned}
& \min \quad \sum_{j=1}^M z_j \\
& \text{u.d.N.} \quad -M_{\text{big}} z_j \leq c_j \leq M_{\text{big}} z_j, \quad j = 1, \dots, M, \\
& \quad \sum_{j=1}^M c_j m_j(x^{(i)}) = 0, \quad i = 1, \dots, N, \\
& \quad b^\top c = 1, \\
& \quad z_j \in \{0, 1\}, \quad j = 1, \dots, M, \\
& \quad c_j \in \mathbb{R}, \quad j = 1, \dots, M.
\end{aligned} \tag{3.4}$$

In praktischen Anwendungen liegen Datenpunkte selten exakt auf einer algebraischen Varietät [4]. Messfehler, numerische Rundungsfehler, Approximationen in vorgelagerten Schätzverfahren sowie Modellfehler treten auf. Dies führt dazu, dass beobachtete Punkte nur in einer Umgebung der zugrunde liegenden Varietät liegen. Aus diesem Grund ist es in naturwissenschaftlichen und technischen Anwendungsbereichen von Bedeutung, numerische Verfahren unter Rauschen zu testen. Im Fall von Rauschen in den Eingabedaten kann die zweite Nebenbedingung abgeschwächt werden zu

$$\begin{aligned}
-\varepsilon_i &\leq \sum_{j=1}^M c_j m_j(x^{(i)}) \leq \varepsilon_i, \quad i = 1, \dots, N, \\
\sum_{i=1}^N \varepsilon_i &\leq \varepsilon \\
\varepsilon_i &\in \mathbb{R}_+, \quad i = 1, \dots, N.
\end{aligned} \tag{3.5}$$

Dies entspricht einer ℓ_1 -Budgetierung der Residuen.

Die erste Nebenbedingung in Verbindung mit der Summe in der Zielfunktion dient der Erzeugung einer sparsen Lösung. Die Big-M-Methode funktioniert dabei wie folgt: Die binären Variablen z_j fungieren als Indikatoren für die Auswahl der Monome. Gilt $z_j = 0$, so folgt $c_j = 0$, und das j -te Monom wird nicht verwendet. Ist $z_j = 1$, so gilt $c_j \in [-M_{\text{big}}, M_{\text{big}}]$, d. h. das Monom kann einen nichtverschwindenden Koeffizienten besitzen, der durch $-M_{\text{big}}$ nach unten und M_{big} nach oben beschränkt ist. Für hinreichend große M_{big} ist die

Formulierung deshalb äquivalent zu (3.3). Diese Formulierung ist keine direkte Kopplung der Binärvariablen z_j und der Polynomkoeffizienten c_j . Die erste Nebenbedingung wird nicht verletzt, wenn $z_j = 1$ aber $c_j = 0$ gewählt wird. Da jedoch die Aktivierung von $z_j = 1$ über den Term $\sum_j z_j$ in der Zielfunktion bestraft wird, wählt der Optimierer $z_j = 1$ nur dann, wenn der zugehörige Koeffizient eine signifikante Verbesserung der Approximation bewirkt. Die erste Summe in der Zielfunktion wirkt somit als Regularisierungsterm, der die Anzahl der verwendeten Monome kontrolliert.

Bemerkung 3.2. Wir haben in der Modellierung des Optimierungsproblems einige Entscheidungen getroffen, zu denen es unterschiedliche Möglichkeiten gibt. Die Formulierung über die Big-M-Bedingung ist nicht die einzige Möglichkeit das Problem (3.3) zu formulieren. Eine andere Möglichkeit wäre zum Beispiel eine Formulierung mithilfe der sogenannten Special Ordered Sets, die zuerst in [1] eingeführt und mittlerweile eine Standardmethode geworden sind [36]. Da diese Formulierung allerdings nicht von allen gängigen Solvern unterstützt wird, haben wir uns in dieser Arbeit für die Formulierung mithilfe der Big-M-Bedingung entschieden.

Bei der Modellierung von Rauschen in den Eingabedaten hätte es mehrere andere Möglichkeiten gegeben. Eine Möglichkeit wäre, statt eines Fehlerbudgets ε den Fehlerterm $\sum_{i=1}^N \varepsilon_i$ mit in die Zielfunktion aufzunehmen. Diese Möglichkeit hätte den Vorteil, dass ε nicht a priori bekannt sein muss, sondern sich den Daten anpasst. Diese Möglichkeit ist vor allem dann sinnvoll, wenn das Rauschen in den samples nicht vor der Optimierung abschätzbar ist. Wir haben uns für ε als Nebenbedingung entschieden, da wir sonst zwei konkurrierende Terme in der Zielfunktion haben. Dies erzeugt das Problem der Gewichtung beider Terme. Auf einen weiteren Grund für unsere Entscheidung gehen wir in Abschnitt 4.5 erneut ein. Eine andere Möglichkeit wäre statt eines gesamten Fehlerbudgets ε , ein Fehlerbudget pro Punkt durch die Nebenbedingungen

$$\begin{aligned}
 -\varepsilon &\leq \sum_{j=1}^M c_j m_j(x^{(i)}) \leq \varepsilon, & i = 1, \dots, N, \\
 \varepsilon &\in \mathbb{R}_+,
 \end{aligned}$$

einzuführen. Dies führt aber dazu, dass einzelne Datenpunkte mit sehr starkem Rauschen eine große Bedeutung für die letztendliche Beschreibung von (3.1) spielen.

Das Ausschließen der Skalierungsinvarianz und der Nulllösung modelliert über den Vektor $b \in \mathbb{R}^M$ könnte auch durch eine Normierung auf $\|c\|_2 = 1$ oder $\sum_{j=1}^M c_j = 1$ ersetzt werden. Dies hätte den Vorteil, dass keine passende Wahl für b im Vorhinein bedacht werden muss. Der

Nachteil besteht allerdings bei $\|c\|_2 = 1$, dass die Problemstruktur nicht mehr ein MILP wäre. Mit unserer Formulierung bewahren wir uns außerdem eine gewisse Flexibilität. Beispielsweise ist damit sowohl $\sum_{j=1}^M c_j = 1$ modellierbar als auch das Fixieren einer Koordinate.

Eine weitere Modellierungsentscheidung betrifft kontinuierliche Koeffizienten.

Bemerkung 3.3. Wir haben sowohl das konzeptionelle Optimierungsproblem als auch das MILP mit kontinuierlichen Koeffizienten $c_j \in \mathbb{R}$ formuliert. In einigen Situationen kann es von Vorteil sein, wenn wir $c_j \in \mathbb{Z}$ wählen. Wir vergleichen im Kapitel 6 zu den numerischen Experimenten beide Varianten miteinander. Eine wichtige Einschränkung betrifft jedoch die Normierungsbedingung $b^\top c = 1$. Wählen wir z.B. $b = e_k$, so kann die Polynomgleichung im kontinuierlichen Fall entsprechend skaliert werden. Dadurch genügt zur Erfüllung dieser Nebenbedingung $c_k \neq 0$. Im ganzzahligen Fall ist dies dagegen nur möglich, wenn die gesuchte Gleichung eine ganzzahlige Darstellung besitzt, in der $c_k = 1$ ist. Eine beliebige Skalierung ist nicht unbedingt möglich. Dies schränkt die Menge der darstellbaren Lösungen deutlich stärker ein, als im kontinuierlichen Fall. In dieser Arbeit verwenden wir die ganzzahlige Variante deshalb nur für Problemklassen, bei denen eine Darstellung mit kleinen ganzzahligen Koeffizienten und einem geeigneten Koeffizienten mit Betrag 1 erwartet wird.

Da wir es im späteren Verlauf der Arbeit erneut betrachten, formulieren wir das Problem mithilfe der ℓ_1 -Norm nun auch.

Problem 3.4. Das Optimierungsproblem zur Optimierung der Parameter einer Polynomgleichung zur Beschreibung von Varietäten mit der ℓ_1 -Norm ist für ein festes $b \in \mathbb{R}^M$ gegeben durch

$$\begin{aligned} \min_{c \in \mathbb{R}^M} \quad & \|c\|_1 \\ \text{s.t.} \quad & \sum_{j=1}^M c_j m_j(x^{(i)}) = 0, \quad i = 1, \dots, N, \\ & b^\top c = 1. \end{aligned} \tag{3.6}$$

Im Fall von Rauschen in den Eingabedaten kann, analog zum ℓ_0 -Problem, die Verallgemeinerung (3.5) genutzt werden. Das Problem kann als LP formuliert werden indem $c = c^+ - c^-$ für $c^+, c^- \in \mathbb{R}_+$ gesetzt wird. Damit ist das ℓ_1 Problem zum Beispiel mit der Ellipsoid-Methode in polynomieller Zeit lösbar [16]. Damit ist es, wie wir später sehen werden, deutlich weniger komplex als das ℓ_0 Problem. Wir betrachten in unseren numerischen Experimenten ein Beispiel eines Schnittpunkts einer Gerade mit dem Einheitskreis.

Anhand dieses Beispiels erkennen wir, dass für unsere Problemklasse im Allgemeinen die Lösung des ℓ_0 - und des ℓ_1 -Problems nicht übereinstimmt.

Als Basis für spätere Resultate nutzen wir den folgenden Satz.

Satz 3.5. *Die Anzahl der Monome in $\mathcal{M}$ hängt von der Dimension d und dem Maximalgrad D ab und ergibt sich zu*

$$M = |\mathcal{M}| = \binom{d+D}{D}.$$

Beweis. Jedes Monom in d Variablen von Grad $\leq D$ hat die Form $x_1^{\alpha_1} \dots x_d^{\alpha_d}$ mit nichtnegativen $\alpha_i \geq 0$. Die Frage nach der Anzahl der Monome lässt sich also zurückführen auf die Frage wie viele Lösungen die Gleichung

$$\alpha_1 + \dots + \alpha_d \leq D$$

in $\mathbb{N}$ hat. Wir machen aus der Ungleichung eine Gleichung indem wir eine weitere Variable α_{d+1} hinzufügen, sodass $\alpha_1 + \dots + \alpha_d + \alpha_{d+1} = D$. Mit der kombinatorischen Formel Stars und Bars [9] ist die Anzahl der Lösungen für die Gleichung und damit die Anzahl der Monome gegeben durch

$$\binom{D + (d+1) - 1}{(d+1) - 1} = \binom{D+d}{d}.$$

□

3.2 Herleitung des Optimierungsproblems für allgemeine Varietäten

In diesem Abschnitt verallgemeinern wir das Optimierungsproblem 3.1 von Hyperflächen auf allgemeine Varietäten. Wir nehmen also an, dass

$$X = \{x^{(1)}, \dots, x^{(N)}\} \subseteq \mathbb{R}^d$$

Stichproben einer allgemeinen Varietät sind. Dies bedeutet, dass die Menge

$$\mathcal{I}(X) \cap \mathbb{R}[x_1, \dots, x_d]_{\leq D}$$

nicht mehr durch eine einzelne Polynomgleichung eindeutig charakterisiert werden kann. Wir suchen stattdessen nach mehreren sparsen Polynomgleichungen, deren gemeinsame Nullstellenmenge die Menge (3.1) approximiert. Hierbei diskutieren wir zwei verschiedene Varianten. Beide haben gemeinsam, dass sie sequentiell ähnlich strukturierte Probleme lösen.

3.2.1 Optimierungsproblem für allgemeine Varietäten - Pivot-Methode

Die Idee ist nun, aus dem Optimierungsproblem, das wir im letzten Abschnitt betrachtet haben, ein sequentielles Verfahren abzuleiten. Wir lösen also mehrmals ein ähnliches Problem wie Problem 3.1 und in jedem Schritt erhalten wir eine Polynomgleichung, die auf den Daten nahezu verschwindet. Dazu müssen wir allerdings garantieren, dass wir nicht mehrmals die gleiche Polynomgleichung erhalten. Dies erreichen wir dadurch, dass wir lineare Unabhängigkeit der Lösungskoeffizienten in unser Modell integrieren. Dazu erzeugen wir in der Koeffizientenmatrix der einzelnen Polynomgleichungen eine Dreiecksstruktur. Wir formulieren nun das sequentielle Optimierungsproblem. Anschließend erläutern wir, wie die Dreiecksstruktur erzeugt wird.

Problem 3.6. Für Iteration $l = 1, \dots$ wähle ein Pivot-Element $c_{p_l}^{(l)}$. Dann sei das sequentielle Optimierungsproblem zur Charakterisierung allgemeiner Varietäten anhand der Nullstellenmengen von ℓ_0 -sparsen Polynomgleichungen in der l -ten Iteration gegeben durch

$$\begin{aligned}
 \min \quad & \sum_{j=1}^M z_j \\
 \text{u.d.N.} \quad & -M_{\text{big}} z_j \leq c_j^{(l)} \leq M_{\text{big}} z_j, & j = 1, \dots, M, \\
 & \sum_{j=1}^M c_j^{(l)} m_j(x^{(i)}) = 0, & i = 1, \dots, N, \\
 & c_{p_l}^{(l)} = 1, \\
 & c_{p_k}^{(l)} = 0, & \text{für } k < l, \\
 & z_j \in \{0, 1\}, & j = 1, \dots, M, \\
 & c_j^{(l)} \in \mathbb{R}, & j = 1, \dots, M.
 \end{aligned}$$

Mit diesem Optimierungsproblem behandeln wir den Fall, dass die Daten rauschfrei von einer Varietät stammen. Analog zum letzten Abschnitt (3.5), lässt sich das Problem ohne Einschränkung anhand eines Gesamtfehlerkontingents auf Daten mit Rauschen verallgemeinern. Ebenso können die Koeffizienten c_j entweder kontinuierlich oder ganzzahlig gewählt werden. Im ganzzahligen Fall ist jedoch, wie in Bemerkung 3.3 angemerkt, die zusätzliche Einschränkung durch die Pivotnormierung $c_{p_l}^{(l)} = 1$ zu beachten. Durch die Hinzunahme der Nebenbedingung drei und vier erzwingen wir ähnlich wie beim Gauß-Algorithmus eine obere Dreiecksstruktur in den Koeffizienten der einzelnen Polynomgleichungen, das heißt wir erhalten eine Koeffizientenmatrix der Art

$$\begin{pmatrix} 1 & * & * & \cdots & * & \cdots & * \\ 0 & 1 & * & \cdots & * & \cdots & * \\ 0 & 0 & 1 & \cdots & * & \cdots & * \\ \vdots & \vdots & \vdots & \ddots & * & \cdots & * \\ 0 & 0 & 0 & \cdots & 1 & \cdots & * \end{pmatrix}.$$

In der Koeffizientenmatrix entspricht eine Zeile einer Polynomgleichung. Durch die obere Dreiecksstruktur erzwingen wir lineare Unabhängigkeit der einzelnen Polynomgleichungen. Wir gehen im Laufe dieser Arbeit auf eine Abbruchbedingung für dieses Verfahren ein.

Zwar erzwingen wir mit diesem Ansatz lineare Unabhängigkeit, allerdings können wir im Allgemeinen nicht erwarten, dass wir damit auch die Menge (3.1) erzeugen. Das folgende Beispiel zeigt, dass wir mit diesem Ansatz nicht unbedingt alle Polynomgleichungen in (3.1) finden.

Beispiel 3.7. Sei im $\mathbb{R}^4$ eine algebraische Varietät bestimmt durch die Nullstellenmengen der Gleichungen

$$f_1 = 1 + x_1 + x_2, \text{ und } f_2 = 1 + x_3 + x_4.$$

Angenommen X sei so gewählt, dass wir theoretisch beide Gleichungen identifizieren können. Wir verwenden die Monombasis

$$\{1, x_1, x_2, x_3, x_4\}.$$

Verwenden wir nun als ersten Pivot $c_1^{(1)}$, den zu Monom 1 gehörigen Pivot, so erhalten wir im ersten Optimierungsschritt eine der beiden Polynomgleichungen f_1 oder f_2 . Im zweiten Optimierungsschritt erzwingen wir jedoch $c_1^{(2)} = 0$. Somit erhalten wir die andere Gleichung nicht und erhalten auch nicht alle Gleichungen in (3.1). Hätten wir im ersten Schritt ein beliebiges anderes Pivotelement gewählt, hätten wir möglicherweise beide Gleichungen bestimmen können.

Dieses Beispiel illustriert, dass die Wahl des Pivotelements eine entscheidende Rolle spielt, um möglichst viele Gleichungen in (3.1) zu bestimmen. Wird ein Monom, das in vielen Gleichungen vorkommt, zu früh gewählt, so können unter Umständen nicht alle Gleichungen erzeugt werden. Dies hat zur Folge, dass wir nicht garantieren können, alle Gleichungen in (3.1) zu bestimmen. Falls dies das Ziel ist, muss auf die Verfahren aus [4] zurückgegriffen werden. Unser Optimierungsproblem hingegen findet sparse interpretierbare Mengen an Gleichungen in (3.1), statt einer garantiert vollständigen Idealbasis. Wir gehen im weiteren Verlauf dieser Arbeit auf eine Wahl der Pivotelemente ein. Hierfür benötigen wir jedoch zunächst die theoretische Analyse des Lösungsraums in Abschnitt 4.1.

3.2.2 Optimierungsproblem für allgemeine Varietäten - No-Good-Cut-Methode

Als Alternative zum sequentiellen Verfahren mit festgesetzten Nullkoeffizienten betrachten wir eine Variante, bei der bereits gefundene Supportmuster durch sogenannte No-Good-Cuts ausgeschlossen werden. Mit den No-Good-Cuts greifen wir eine Idee auf, die beispielsweise in [27] umgesetzt wurde, um bereits bekannte Lösungen auszuschließen. Ziel ist es, mehrere sparse Polynome zu bestimmen, ohne dabei die späteren Teilprobleme durch eine explizite Fixierung früher verwendeter Pivot-Monome strukturell einzuschränken.

Ausgangspunkt ist zunächst erneut das Problem 3.1. Im ersten Schritt wird das Problem zunächst einmalig gelöst, daraufhin werden die bisher gefundenen Supportmuster ausgeschlossen. Sei dazu $z^{(r)} \in \{0, 1\}^M$ das in Iteration r gefundene Aktivierungsmuster der Monome und

$$S_1^{(r)} := \{j : z_j^{(r)} = 1\}, \quad S_0^{(r)} := \{j : z_j^{(r)} = 0\}.$$

Dann wird für alle nachfolgenden Optimierungsprobleme die No-Good-Ungleichung

$$\sum_{j \in S_1^{(r)}} (1 - z_j) + \sum_{j \in S_0^{(r)}} z_j \geq 1$$

hinzugefügt. Diese Bedingung stellt sicher, dass mindestens eine Binärvariable ihren Wert gegenüber $z^{(r)}$ ändern muss. Somit verhindern wir, dass wir in einer nachfolgenden Operation exakt dasselbe Supportmuster erhalten. Dies wird wiederholt, bis keine neue zulässige Gleichung mehr gefunden wird oder die gewünschte Anzahl von Gleichungen erreicht ist. Wir erhalten damit das folgende Optimierungsproblem:

Problem 3.8. Für $l = 1, \dots$ wähle ein Pivot-Element $c_{p_l}^{(l)}$. Dann sei das sequentielle Optimierungsproblem zur Charakterisierung allgemeiner Varietäten in der No-Good-Cut-Variante gegeben durch

$$\begin{aligned}
& \min \quad \sum_{j=1}^M z_j \\
& \text{u.d.N.} \quad -M_{\text{big}} z_j \leq c_j^{(l)} \leq M_{\text{big}} z_j, & j = 1, \dots, M, \\
& \quad \sum_{j=1}^M c_j^{(l)} m_j(x^{(i)}) = 0, & i = 1, \dots, N, \\
& \quad c_{p_l}^{(l)} = 1, \\
& \quad \sum_{j \in S_1^{(r)}} (1 - z_j) + \sum_{j \in S_0^{(r)}} z_j \geq 1, & r = 1, \dots, l-1, \\
& \quad z_j \in \{0, 1\}, & j = 1, \dots, M, \\
& \quad c_j^{(l)} \in \mathbb{R}, & j = 1, \dots, M.
\end{aligned}$$

Im Unterschied zur Pivot-Methode werden vorher verwendete Pivotkoeffizienten in späteren Optimierungsiterationen nicht ausgeschlossen. Stattdessen werden nur identische Supportmuster ausgeschlossen. Dies ist vor allem dann von Vorteil, wenn mehrere relevante Gleichungen zur Beschreibung einer Varietät überschneidende Supportstrukturen haben. Ein weiterer Vorteil der No-Good-Cut-Methode ist, dass das Verfahren näher an der ursprünglichen Problemstellung bleibt. Die Pivot-Methode erzwingt lineare Unabhängigkeit der gefundenen Gleichungen durch eine zusätzliche strukturelle Einschränkung der Koeffizientenmatrix. Diese Einschränkung ist für den Algorithmus nützlich, kann aber dazu führen, dass relevante Gleichungen frühzeitig ausgeschlossen werden.

Der wesentliche Nachteil besteht darin, dass wir mit der No-Good-Cut-Variante keine lineare Unabhängigkeit garantieren, sondern nur identische Supportmuster ausschließen. Weiterhin wird bei der Pivot-Methode der zulässige Bereich durch zusätzliche Fixierungen strukturell kleiner. Bei der No-Good-Cut-Variante wird dagegen pro Iteration eine zusätzliche Ausschlussbedingung hinzugefügt, sodass das Modell zwar andere Lösungen ausschließt, aber zugleich durch die wachsende Zahl an Nebenbedingungen immer



restriktiver und schwieriger Lösbar wird. Somit kann die No-Good-Cut-Methode bei hochdimensionalen Varietäten in späteren Iterationen rechenintensiver werden. In Kapitel 6 vergleichen wir die beiden Verfahren auch hinsichtlich der Rechenkomplexität miteinander.

4 Komplexität des Optimierungsproblems

In diesem Abschnitt analysieren wir die Komplexität des Optimierungsproblem (3.1). Dazu betrachten wir zunächst den rauschfreien Fall. Wir nutzen dazu Standardresultate der linearen Algebra, um dadurch auf Kriterien für die Existenz und Eindeutigkeit einer Lösung zu schließen. Nach dieser Analyse des Lösungsraums können wir auf die Wahl der Pivotelemente in unseren sequentiellen Optimierungsproblemen eingehen. Daraufhin zeigen wir, dass unser Optimierungsproblem NP-schwer ist, um danach Kriterien und Presolvingschritte herzuleiten, die die Komplexität des Problems verringern.

4.1 Struktur des Lösungsraums

In diesem Abschnitt wiederholen wir einige Resultate der linearen Algebra aus [15], um diese Resultate auf unser Problem anzuwenden und damit den Lösungsraum unseres Optimierungsproblems genauer beschreiben zu können. Hierfür definieren wir zunächst den Begriff der Evaluationsmatrix.

Definition 4.1. Wir bezeichnen die Matrix gegeben durch

$$A \in \mathbb{R}^{N \times M}, \quad A_{ij} := m_j(x^{(i)}).$$

als Evaluationsmatrix.

Jede Spalte der Evaluationsmatrix entspricht dabei einem der ausgewählten Monome, während die Zeilen mit den Stichprobenpunkten korrespondieren. Seien also zum Beispiel $\{x^{(1)}, \dots, x^{(N)}\} \subset \mathbb{R}$ und Monome $\{1, x, x^2, \dots, x^n\}$ dann ist die Evaluationsmatrix

gegeben durch

$$A = \begin{pmatrix} 1 & x^{(1)} & (x^{(1)})^2 & \dots & (x^{(1)})^n \\ 1 & x^{(2)} & (x^{(2)})^2 & \dots & (x^{(2)})^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x^{(N)} & (x^{(N)})^2 & \dots & (x^{(N)})^n \end{pmatrix}.$$

Im Folgenden analysieren wir die Struktur der Evaluationsmatrix genauer, um damit Rückschlüsse auf die Lösbarkeit unseres Optimierungsproblems zu ziehen. Wir wiederholen weiterhin die Definition des Kerns einer Matrix aus [15], da wir anhand dieser Definition direkt auf ein Kriterium zur Existenz einer Polynomgleichung zur Beschreibung der Datenpunkte schließen können.

Definition 4.2. Sei $A \in \mathbb{R}^{m \times n}$. Dann ist der Kern von A gegeben durch

$$\ker(A) = \{x \in \mathbb{R}^n \mid Ax = 0\}.$$

Wir erinnern zusätzlich an einen Satz aus der linearen Algebra, der einen Zusammenhang zwischen dem Rang einer Matrix und der Kerndimension herstellt.

Satz 4.3. Sei $A \in \mathbb{R}^{m \times n}$ mit Rang $\text{rank}(A)$. Dann gilt

$$\text{rank}(A) + \dim(\ker(A)) = n. \quad (4.1)$$

Beweis. Siehe [15]. □

Nun haben wir die Grundlagen geschaffen, um auf ein Kriterium zur Existenz einer Lösung für unser Optimierungsproblem 3.1 zu schließen.

Satz 4.4. Seien

$$X = \{x^{(1)}, \dots, x^{(N)}\} \subset \mathbb{R}^d, \quad d \in \mathbb{N}. \quad (4.2)$$

exakte Stichproben einer algebraischen Varietät. Seien weiter $\{m_1, \dots, m_M\}$ die Monombasis aller Monome in d Variablen mit $\text{Grad} \leq D$ und $A \in \mathbb{R}^{N \times M}$ die zu den Stichproben und Monomen gehörige Evaluationsmatrix.

Es existiert eine nichttriviale Lösung für das Optimierungsproblem 3.1 genau dann, wenn ein $c \in \ker(A)$ existiert mit $b^\top c \neq 0$.

Beweis. „ $\Rightarrow$ “: Sei $\hat{c} = (c_1, \dots, c_M) \in \mathbb{R}^M$ eine nichttriviale Lösung des Optimierungsproblems (3.1). Dann ist die zweite Nebenbedingung äquivalent zu $Ac = 0$. Zusammen mit der dritten Nebenbedingung und Definition 4.2 folgt die Aussage.

„ $\Leftarrow$ “: Sei ein $c \in \ker(A)$ mit $b^\top c \neq 0$. Aufgrund von Definition 4.2 ist die zweite Nebenbedingung erfüllt. Ebenso ist die dritte Nebenbedingung aufgrund von $b^\top c \neq 0$ durch entsprechende Skalierung von c erfüllbar. Für die erste Nebenbedingung für alle j mit $c_j \neq 0$ wähle $z_j = 1$ und für die sonstigen Indizes wähle $z_j = 0$. Somit existiert ein hinreichend großes M_{big} mit

$$|c_j| \leq M_{\text{big}} z_j$$

Somit existiert eine zulässige Lösung von Problem (3.1). $\square$

Dieser Existenzsatz gilt sowohl für Problem 3.1 als auch in der ersten Iteration für die sequentiellen Optimierungsprobleme. In weiteren Iterationen gilt er weiter, wobei die Bedingung $b^\top c \neq 0$ durch die jeweiligen Pivotbedingungen ergänzt werden muss. Allerdings ist dieser Existenzsatz abhängig von einigen Modellierungsentscheidungen, die wir in Kapitel 3 getroffen haben.

Bemerkung 4.5. *Es wäre auch möglich gewesen die sparsity in Problem 3.1 als harten Constraint in die Nebenbedingungen mit aufzunehmen, statt fehlende sparsity in der Zielfunktion zu bestrafen. Dadurch würde allerdings der Existenzsatz seine Gültigkeit verlieren, da er keine Aussage über die Existenz einer sparsen Lösung mit höchstens s Koeffizienten ungleich 0 macht. In diesem Fall muss geprüft werden, ob eine solche Lösung im Kern der Evaluationsmatrix existiert.*

Weiterhin haben wir Problem 3.1 mit kontinuierlichen Parametern formuliert. Da wir uns in unseren numerischen Experimenten allerdings auch ganzzahlige Koeffizienten ansehen, müssen wir an dieser Stelle anmerken, dass in diesem Fall die Bedingung $b^\top c \neq 0$ nicht genügt. Stattdessen muss eine ganzzahlige Lösung mit $b^\top c = 1$ existieren.

Nachdem wir nun eine Aussage über die Existenz einer Polynomgleichung zur Beschreibung der Stichprobenpunkte machen können, erhalten wir mit dem folgenden Satz ein Kriterium zur Eindeutigkeit dieser Lösung.

Satz 4.6. *Seien*

$$X = \{x^{(1)}, \dots, x^{(N)}\} \subset \mathbb{R}^d, \quad d \in \mathbb{N}. \quad (4.3)$$

Daten auf einer Varietät ohne Rauschen. Seien weiter m_j , $j \in \mathbb{N}$ vorher definierte Monome und $A \in \mathbb{R}^{N \times M}$ die Evaluationsmatrix. Gilt nun

$$\dim(\ker(A)) = 1, \quad (4.4)$$

so existiert bis auf Skalierung ein eindeutiges $c \in \ker(A)$. Existiert nun eine Lösung für Optimierungsproblem 3.1, also ist zusätzlich $b^\top c \neq 0$, so ist diese Lösung eindeutig.

Für den Beweis dieses Satzes wiederholen wir ein Lemma aus der linearen Algebra.

Lemma 4.7. Jeder eindimensionale Untervektorraum $V \subseteq \mathbb{R}^n$ hat die Form $V = \{\lambda v \mid \lambda \in \mathbb{R}\}$ für jedes festes $v \in V$ mit $v \neq 0$.

Beweis. Siehe [15]. □

Beweis von Satz 4.6. Es sei $\dim(\ker(A)) = 1$, dann ist $\ker(A)$ ein eindimensionaler Untervektorraum des $\mathbb{R}^M$. Mit Lemma 4.7 gilt für jedes $c \in \ker(A)$ mit $c \neq 0$ die Eigenschaft

$$\ker(A) = \{\lambda c \mid \lambda \in \mathbb{R}\}$$

Sei nun $c' \in \ker(A)$ dann existiert $\lambda \in \mathbb{R}$ mit $c' = \lambda c$. Damit sind die zugehörigen Polynome f und g verbunden durch $g = \lambda f$. Somit sind die Lösungen eindeutig bis auf Skalierung. Aufgrund der Bedingung $b^\top c = 1$ verschwindet diese Skalierungsfreiheit und die Lösung des Optimierungsproblems ist eindeutig. □

Wir erhalten somit durch die Analyse des Untervektorraums $\ker(A)$ ein Kriterium für die Existenz und die Eindeutigkeit einer Lösung im Fall von Hyperflächen und im Fall, dass kein Rauschen in den Eingabedaten existiert. Im Fall von allgemeinen Varietäten, lässt sich folgende Aussage treffen.

Satz 4.8. Sei $A \in \mathbb{R}^{N \times M}$ mit $t = \dim(\ker(A))$. Dann existieren höchstens t linear unabhängige Polynome in

$$\mathcal{I}(X) \cap \mathbb{R}[x_1, \dots, x_d]_{\leq D}.$$

Insbesondere können die sequentiellen Optimierungsprobleme (3.6) und (3.8) höchstens t linear unabhängige Lösungen liefern.

Beweis. Dies folgt, da die Koeffizientenvektoren aller solchen Polynome in $\ker(A)$ liegen. Ein t -dimensionaler Raum enthält höchstens t linear unabhängige Vektoren. □

Bemerkung 4.9. Aus den bisher getroffenen Aussagen folgt, dass ein hochdimensionaler Kern zu einem großen Lösungsraum führt, während ein niedrigdimensionaler Kern einen kleinen Lösungsraum impliziert. Aufgrund von Satz 4.3 folgt, dass der Lösungsraum groß ist, wenn die Evaluationsmatrix stark rangdefizient ist. Dies gibt auch einen Indikator dafür, wie viele linear unabhängige Gleichungen wir mit Problem 3.6 finden können: Die Kerndimension gibt uns die Anzahl der maximal möglichen linear unabhängigen Gleichungen im Nullraum von A und somit auch die Anzahl der maximal aufspürbaren Gleichungen mit Problem 3.6.

Wir können nun eine einfache Schlussfolgerung aus den bisher bekannten Resultaten ziehen und damit die Anzahl der benötigten Stichproben zum eindeutigen Bestimmen einer Varietätengleichung herleiten.

Korollar 4.10. Seien $d \in \mathbb{N}$, $D \in \mathbb{N}$, $D \geq 0$ und

$$\mathcal{M} = \{x_1^{\alpha_1} x_2^{\alpha_2} \cdots x_d^{\alpha_d} \mid \alpha_1 + \cdots + \alpha_d \leq D\}$$

die Menge aller Monome in d Variablen von Maximalgrad D . Dann sind mindestens

$$\binom{d+D}{D} - 1$$

notwendig, um $\mathcal{I}(X) \cap \mathbb{R}[x_1, \dots, x_d]_{\leq D}$ bis auf Skalierung eindeutig zu bestimmen. Diese Anzahl ist hinreichend für Eindeutigkeit, wenn zusätzlich

$$\text{rank}(A) = \binom{d+D}{D} - 1.$$

Beweis. Dies folgt direkt aus Satz 3.5 in Zusammenhang mit Satz 4.6. □

Bemerkung 4.11. Die notwendige Bedingung für Eindeutigkeit bis auf Skalierung gilt für die Menge $\mathcal{I}(X) \cap \mathbb{R}[x_1, \dots, x_d]_{\leq D}$, aber nicht unbedingt für das Optimierungsproblem 3.1. Hier können die sparsity und im Fall von Ganzzahligkeit der eingeschränkte Lösungsraum dazu führen, dass auch weniger Stichproben genügen. Wir analysieren dies in unseren numerischen Experimenten.

Bisher haben wir uns in der Existenz- und Eindeutigkeitsanalyse auf den Fall konzentriert, bei dem kein Rauschen in den Eingabedaten herrscht. Da dies nicht der Regelfall ist und bei samples in der realen Welt häufig Rauschen auftritt, besprechen wir im Folgenden ein Konzept, mit dem wir das Ganze auf den Fall mit Rauschen verallgemeinern können.

Das Problem bei verrauschten Daten ist, dass wir uns auf das Konzept der linearen Abhängigkeit nicht mehr verlassen können. Es ist üblich, dass Daten, die ohne Rauschen linear abhängig sind, durch ein Hinzufügen von Rauschen linear unabhängig werden. Dies macht die bisherigen Analysen des Lösungsraums auf Basis des Kerns der Evaluationsmatrix zunächst unbrauchbar. Um dieses Problem in den Griff zu bekommen, führen wir mit der Singulärwertzerlegung ein Konzept aus der numerischen Mathematik ein. Mit diesem Konzept können wir Daten erkennen, die 'beinahe linear abhängig' sind. Somit können wir auch bei verrauschten Daten den numerischen Rang der Evaluationsmatrix abschätzen.

Als Basis für die Singulärwertzerlegung erinnern wir an folgende Definition aus der linearen Algebra [15].

Definition 4.12. Eine Matrix $U \in \mathbb{R}^{n \times n}$ heißt orthogonal, wenn $U^\top U = I_n$.

Damit können wir folgenden Satz aus [15] zitieren.

Satz 4.13 (Singulärwertzerlegung - reeller Fall). Sei $A \in \mathbb{R}^{n \times m}$ mit $q = \min\{m, n\}$ und $\text{rank}(A) = r$. Dann gibt es orthogonale Matrizen $U \in \mathbb{R}^{n \times n}$, $V \in \mathbb{R}^{m \times m}$ und eine quadratische Diagonalmatrix

$$\Sigma_q = \begin{pmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & \sigma_q \end{pmatrix},$$

sodass $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0 = \sigma_{r+1} = \cdots = \sigma_q$ und $A = U \Sigma V^\top$. Hierbei gilt

$$\begin{aligned} \Sigma &= \Sigma_q && \text{für } m = n, \\ \Sigma &= \begin{pmatrix} \Sigma_q & 0 \end{pmatrix} && \text{für } m > n, \\ \Sigma &= \begin{pmatrix} \Sigma_q \\ 0 \end{pmatrix} && \text{für } m < n. \end{aligned} \tag{4.5}$$

Die Parameter $\sigma_1, \dots, \sigma_q$ sind die absteigend sortierten, Quadratwurzeln der Eigenwerte von $A^\top A$ bzw. $AA^\top$.

Beweis. Siehe [15]

□

Wie zum Beispiel in [5] argumentiert, kann man anhand der Singulärwertzerlegung ablesen, dass Richtungen einer Matrix bis auf ein Rauschen linear abhängig sind. Ein kleiner Singulärwert bedeutet, dass es eine Linearkombination der Spalten gibt, die nahezu verschwindet. Die Idee zur Abschätzung des Rangs der Evaluationsmatrix ist also zunächst einen Schwellwert für die Singulärwerte zu definieren. In der Praxis wird der Rang oft mit einer relativen Schwelle bestimmt, zum Beispiel

$$\frac{\sigma_i}{\sigma_1} < \varepsilon,$$

wobei ε typischerweise zwischen 10^{-12} und 10^{-6} liegt [10]. Die Anzahl der Singulärwerte, die oberhalb dieses Schwellwerts liegen, werden als Richtwert für den Rang der Matrix genommen. Damit folgt die Dimension des Kerns aus der Formel aus Satz 4.3. Wir sprechen im weiteren Verlauf dieser Arbeit noch häufiger über den Nullraum einer Matrix. Im Fall von verrauschten Daten ist dabei der numerische Kern der Matrix gemeint.

4.2 Wahl der Pivotelemente

Im letzten Abschnitt haben wir gesehen, wie der Lösungsraum und der Kern der Evaluationsmatrix miteinander verbunden sind. In diesem Abschnitt nehmen wir dies zum Anlass, um eine Strategie zu entwickeln die Pivotelemente in Problem 3.6 zu wählen, um möglichst viele Gleichungen in

$$\mathcal{I}(X) \cap \mathbb{R}[x_1, \dots, x_d]_{\leq D}$$

zu finden. Sei dazu $t = \dim(\ker(A))$ und sei $N \in \mathbb{R}^{M \times t}$ eine Matrix, deren Spalten eine Orthonormalbasis von $\ker(A)$ bilden. Damit gilt

$$N^\top N = I_t \quad \text{und} \quad \ker(A) = \{Ny \mid y \in \mathbb{R}^t\}.$$

Jeder Koeffizientenvektor $c \in \ker(A)$ kann somit in der Form $c = Ny$ für $y \in \mathbb{R}^t$ dargestellt werden. Für die j -te Koordinate von c gilt dann $c_j = N_{j,:}y$, wobei $N_{j,:} \in \mathbb{R}^{1 \times t}$ die j -te Zeile von N bezeichnet. Die Zeile $N_{j,:}$ beschreibt somit, wie stark und in welche Richtungen sich der Koeffizient des j -ten Monoms innerhalb des Nullraums verändern kann. Zur Bewertung eines möglichen Pivotelements definieren wir

$$\rho_j := \|N_{j,:}\|_2 = \max_{\|y\|_2=1} |N_{j,:}y|, \quad j = 1, \dots, M.$$

Damit beschreibt ρ_j , wie groß der j -te Koeffizient innerhalb einer normierten Nullraumrichtung höchstens werden kann. Ist $\rho_j = 0$, so verschwindet der j -te Koeffizient für jeden Vektor $c \in \ker(A)$. Ein Monom mit $\rho_j = 0$ kann daher nicht als Pivotelement genutzt werden. Ein großer Wert von ρ_j bedeutet dagegen, dass der zugehörige Koeffizient in mindestens einer Nullraumrichtung stark ausgeprägt ist. Der Wert von ρ_j ist dabei unabhängig von der konkreten Wahl der Orthonormalbasis von $\ker(A)$. Ist $\tilde{N}$ eine weitere Orthonormalbasis von $\ker(A)$, dann gilt $\tilde{N} = NQ$, wobei $Q \in \mathbb{R}^{t \times t}$ orthogonal ist. Somit gilt

$$\begin{aligned}\|\tilde{N}_{j,:}\|_2^2 &= \|N_{j,:}Q\|_2^2 \\ &= (N_{j,:}Q)(N_{j,:}Q)^\top \\ &= N_{j,:}QQ^\top N_{j,:}^\top \\ &= N_{j,:}I_t N_{j,:}^\top \\ &= N_{j,:}N_{j,:}^\top = \|N_{j,:}\|_2^2.\end{aligned}$$

Da Normen nichtnegativ sind, folgt daraus

$$\|\tilde{N}_{j,:}\|_2 = \|N_{j,:}\|_2.$$

Zur Bestimmung der Pivots sortieren wir nun die erhaltenen ρ_j Werte absteigend. Große Werte von ρ_j werden zuerst als Pivot gewählt, da sie anschaulich gesprochen die größte Bewegungsfreiheit innerhalb von $\ker(A)$ haben. Es kann vorkommen, dass zwei Monome einen großen Wert ρ_j besitzen, ihre zugehörigen Zeilen in N jedoch linear abhängig sind. In diesem Fall liefern die beiden Koordinaten keine voneinander unabhängigen Informationen über den Nullraum. Dies führt dazu, dass das entstehende Optimierungsproblem 3.6 unzulässig wird. Man könnte durch die Hinzunahme einer Rangbedingung im Pivot-Algorithmus dieses Problem umgehen. Stattdessen haben wir uns dazu entschieden dieses Problem in Kauf zu nehmen und im Fall von Unzulässigkeit zum nächstkleineren ρ_j überzugehen.

Bemerkung 4.14. Diese Pivotregel ist heuristisch motiviert: Sie zielt darauf ab, Monome zu bevorzugen, die innerhalb des Kerns eine große Bewegungsfreiheit haben. Sie garantiert jedoch trotzdem nicht, dass alle Gleichungen in (3.1) gefunden werden. Im Fall, dass ein Pivot ein unzulässiges Problem erzeugt, kann zurück zur Pivotwahl gegangen werden, um den nächstbesseren Pivot zu wählen. Um garantiert alle Gleichungen zu finden, müsste auf Kosten der sparsity wie in [4] auf andere Methoden zurückgegriffen werden.

Nun folgt noch eine kurze Bemerkung zur Bestimmung von N .

Bemerkung 4.15. Für diese Pivotierung benötigen wir eine Matrix N dessen Spalten eine Orthonormalbasis von $\ker(A)$ bildet. Solch eine Matrix kann mit dem Gauß-Algorithmus und dem Gram-Schmidt Verfahren bestimmt werden. Alternativ bilden die Spalten von V in Satz 4.13, die zu den Singulärwerten 0 gehören, ebenso eine Orthonormalbasis von $\ker(A)$. Dies folgt wegen der Definition von V als orthogonale Matrix und den verschwindenden Singulärwerten. Dieses Verfahren kann auch auf beinahe verschwindende Singulärwerte angewandt werden.

4.3 Beweis der NP-Schwere des Problems

In diesem Abschnitt beweisen wir, dass das Entscheidungsproblem zu unserem konzeptionellen (3.3) NP-schwer ist. Somit ist für hinreichend großes M_{big} auch Problem 3.1 NP-schwer. Da Problem 3.6 eine Erweiterung von Problem 3.1 ist, ist damit auch das zugehörige Entscheidungsproblem NP-schwer. Diesen Beweis führen wir, indem wir ein anderes Problem, von dem in [14] die NP-Schwere durch Reduktion von Exact-3-cover-set bewiesen wurde, auf unser Problem reduzieren. Dies ist ein Standardproblem, auf das viele ähnliche sparse Optimierungsprobleme reduziert werden. Dazu definieren wir zunächst das Entscheidungsproblem zu unserem Optimierungsproblem. Wir kürzen das Problem im Folgenden mit SVF (sparse variety fit) ab.

Problem 4.16. Seien $X = \{x^{(1)}, \dots, x^{(N)}\} \subset \mathbb{R}^d$, Monome $\mathcal{M} = \{m_1, \dots, m_M\}$, $\varepsilon \geq 0$ und eine ganze Zahl $k \in \mathbb{N}$ und $b \in \mathbb{R}^M$. Die Frage ist, existiert ein $c \in \mathbb{R}^M$ mit höchstens k Nichtnull-Einträgen, sodass

$$\max_{i=1, \dots, N} \left| \sum_{j=1}^M c_j m_j(x^{(i)}) \right| \leq \varepsilon, \quad b^\top c = 1.$$

Zusätzlich benennen wir das Problem MWSTLE (minimum weight solution to linear equations) welches von Garey und Johnson als NP-schwer klassifiziert wurde [14].

Problem 4.17. Gegeben seien $A \in \mathbb{R}^{N \times (M-1)}$, $y \in \mathbb{R}^N$ und $k \in \mathbb{N}$. Existiert ein Vektor $x \in \mathbb{R}^{M-1}$ mit $\|x\|_0 \leq k$, sodass

$$Ax = y?$$

Wir haben die exakte Variante von Problem 4.17 formuliert. Von Natarajan wurde in [21] auch eine Variante mit Fehlertoleranz $\varepsilon > 0$ als NP-schwer klassifiziert. Wir verwenden zur Reduktion jedoch die exakte Variante, da die Variante von Natarajan den Spezialfall $\varepsilon = 0$ nicht inkludiert. Durch Reduktion von Problem 4.17 auf Problem 4.16 zeigen wir nun, dass das Entscheidungsproblem zu unserem Optimierungsproblem NP-schwer ist.

Satz 4.18. *Das Entscheidungsproblem 4.16 ist NP-schwer.*

Beweis. Es sei eine beliebige Instanz von MWSTLE gegeben. Es genügt in unserem Problem den Spezialfall $\varepsilon = 0$ zu betrachten, da dieser Spezialfall im allgemeinen Entscheidungsproblem 4.16 enthalten ist. Wir konstruieren daraus eine Instanz unseres Entscheidungsproblems wie folgt. Wir setzen

$$d = M.$$

Für jede Zeile $i = 1, \dots, N$ definieren wir einen Punkt

$$x^{(i)} = (A_{i1}, A_{i2}, \dots, A_{iM-1}, -y_i) \in \mathbb{R}^M.$$

Als Monome wählen wir ausschließlich lineare Monome

$$m_j(x) = x_j, \quad j = 1, \dots, M.$$

Weiter wählen wir den Normierungsvektor $b = e_M$, den M -ten Einheitsvektor, und setzen die sparsity-Schranke $K = k + 1$. Wir zeigen nun die Äquivalenz der beiden Probleme. " $\Rightarrow$ " Angenommen, die MWSTLE-Instanz besitzt eine Lösung $\hat{c} \in \mathbb{R}^{M-1}$ mit

$$A\hat{c} = y, \text{ und } \|\hat{c}\|_0 \leq k.$$

Definiere nun

$$c = (\hat{c}, 1) \in \mathbb{R}^M,$$

dann gilt wegen $b = e_M$, dass $b^\top c = c_M = 1$. Weiter gilt für jedes $i = 1, \dots, N$, dass

$$\sum_{j=1}^M c_j m_j(x^{(i)}) = \sum_{j=1}^{M-1} c_j A_{ij} - y_i = (A\hat{c})_i - y_i = 0.$$

Außerdem gilt

$$\|c\|_0 = \|\hat{c}\|_0 + 1 \leq k + 1 = K.$$

Damit ist die konstruierte Instanz des Entscheidungsproblems zulässig.

” $\Leftarrow$ ” Angenommen nun, die konstruierte Instanz des Entscheidungsproblems besitzt eine Lösung

$$c = (c_1, \dots, c_M) \in \mathbb{R}^M$$

mit

$$\|c\|_0 \leq K, \quad b^\top c = 1,$$

sowie

$$\sum_{j=1}^M c_j m_j(x^{(i)}) = 0 \quad \forall i = 1, \dots, N.$$

Da $b = e_M$ gilt, folgt unmittelbar $c_M = 1$. Setze nun

$$\hat{c} = (c_1, \dots, c_{M-1}) \in \mathbb{R}^{M-1}.$$

Dann gilt für jedes $i = 1, \dots, N$, dass

$$0 = \sum_{j=1}^M c_j m_j(x^{(i)}) = \sum_{j=1}^{M-1} c_j A_{ij} - y_i.$$

Somit folgt $A\hat{c} = y$. Da zusätzlich $c_M = 1 \neq 0$ gilt, erhalten wir

$$\|\hat{c}\|_0 = \|c\|_0 - 1 \leq K - 1 = k.$$

Damit besitzt auch die ursprüngliche MWSTLE-Instanz eine zulässige Lösung. Die Konstruktion verwendet lediglich das Hinzufügen einer zusätzlichen Koordinate und kann offensichtlich in polynomialer Zeit durchgeführt werden. Damit ist die gegebene MWSTLE-Instanz genau dann lösbar, wenn die konstruierte SVF-Instanz lösbar ist. Da MWSTLE NP-schwer ist, folgt die NP-Schwere des Entscheidungsproblems. $\square$

Bemerkung 4.19. Der Beweis des letzten Satzes zeigt außerdem, dass nicht ein hoher Maximalgrad der Monome das Problem NP-schwer macht, da das Problem auch schon für $D = 1$ NP-schwer ist. Auch die Varietätenstruktur spielt in der Reduktion keine Rolle. Die NP-Schwere entsteht durch die sparsity-Bedingung. Die konvexe Relaxierung mithilfe der ℓ_1 -Norm ist dagegen als LP formulierbar und somit in polynomialer Zeit lösbar. Wir vergleichen die beiden Verfahren in numerischen Experimenten miteinander.

4.4 Einflussfaktoren auf das Branch-and-Bound Verhalten

Im letzten Abschnitt haben wir gesehen, dass unser Optimierungsproblem NP-schwer ist. Unter der Annahme, dass $P \neq NP$ gibt es keinen Lösungsalgorithmus, der sämtliche Instanzen in polynomieller Laufzeit löst. Einzelne Instanzen des Problems können jedoch trotzdem effizient gelöst werden. Diese Beobachtung nehmen wir zum Anlass, in diesem Abschnitt zu betrachten, welche Faktoren das Verhalten eines Branch-and-Bound-Solvers in der Praxis beeinflussen. Die Laufzeit des Branch-and-Bound Verfahrens hängt vor allem von der Stärke der LP-Relaxationen, der Anzahl und Struktur der ganzzahligen Variablen und effektiven Such- und Branchingstrategien ab [20, 36].

4.4.1 Einflussfaktor Big-M

Ein wesentlicher Faktor für die Qualität der LP-Relaxation, der bereits in gängiger Literatur untersucht wurde [24, 18], ist die Big-M-Kopplung über die Nebenbedingung

$$\begin{aligned} -M_{\text{big}}z_j &\leq c_j \leq M_{\text{big}}z_j \quad \forall j = 1, \dots, M, \\ z_j &\in \{0, 1\} \quad \forall j = 1, \dots, M. \end{aligned} \tag{4.6}$$

Durch die Big-M-Bedingung sind die Binärvariablen mit den kontinuierlichen Variablen gekoppelt. Ein großes M_{big} führt dazu, dass die Relaxationen sehr weit sind, wodurch viele z_j -Variablen Werte zwischen 0 und 1 annehmen. Dies vergrößert den sogenannten Root Relaxation Gap zwischen LP-Relaxationslösung und optimaler ganzzahliger Lösung und führt zu einem tiefen und verzweigten Optimierungsbaum [36]. Dies führt zu einer schlechten Performance des Branch-and-Bound-Solvers [18]. Zugleich kann M_{big} nicht beliebig klein gewählt werden, da es die maximal möglichen Größenordnungen der Koeffizienten darstellt. Ein zu klein gewähltes M_{big} führt also dazu, dass möglicherweise die optimale Lösung abgeschnitten wird. Dadurch entsteht ein Zielkonflikt zwischen Gültigkeit der Modellierung der ℓ_0 -Bedingung und Lösungskomplexität. In Abschnitt 4.5 diskutieren wir Möglichkeiten Tightening durchzuführen, um die von M_{big} erzeugten Schranken vor Beginn der Branch-and-Bound Optimierung zu verbessern, ohne dabei zulässige Lösungen abzuschneiden.

4.4.2 Einflussfaktor Monomanzahl

Die Anzahl der möglichen Monome hat einen entscheidenden Einfluss auf die Komplexität des MILP-Modells und die Performance des Solvers. Mit einer größeren Anzahl von aktiven Monomen erhöhen wir die Wahrscheinlichkeit eine geeignete Polynomgleichung zur Beschreibung von (3.1) zu erhalten. Andererseits steigt mit zunehmender Monomanzahl M zunächst die Anzahl der Binärvariablen z_j im Optimierungsmodell. Aufgrund der sparsity-Bedingung muss der Solver zwischen $\binom{M}{s}$ möglichen Aktivierungsmustern der Monome unterscheiden, um eine genau s -sparse Lösung, also eine Lösung mit s aktiven z_j , zu erzeugen. Dieser kombinatorische Suchraum führt dazu, dass die Größe des Branch-and-Bound Baums im Worst-Case exponentiell mit der Anzahl der Monome wächst [20]. Selbst bei moderaten Werten von M und s wächst diese Zahl schnell. In dieser Arbeit konzentrieren wir uns daher auf niedriggradige Polynome. Bei sehr hochgradigen Polynomen kann es zusätzlich zu dem Problem kommen, dass wir uns eher einer Verzerrung durch Rauschen als der wahren Varietät anpassen.

4.4.3 Einflussfaktor Rang der Evaluationsmatrix

Ein weiterer Einflussfaktor ist der Rang der Evaluationsmatrix. Aus Satz 4.6 und Bemerkung 4.9 folgt, dass eine stark rangdefiziente Matrix und somit eine hohe Kerndimensionalität dazu führt, dass es eine hochdimensionale affine Menge von Lösungen der Gleichung $Ac = 0$ und $b^\top c = 1$ gibt. In der LP-Relaxation des MILP kann dies zu einer starken Degeneration führen: Es können zahlreiche zulässige Lösungen mit ähnlichen Zielfunktionswerten existieren, zwischen denen der Solver kaum unterscheiden kann. Dies kann zu schwachen Bounds in den einzelnen Branches führen. Die Degeneration kann dadurch auch effektive Branching Entscheidungen erschweren, da viele Branches zu Relaxationslösungen führen, die sich nur geringfügig unterscheiden. Die Auswahl effektiver Branching-Strategien kann einen erheblichen Einfluss auf die Lösungszeit haben [20]. Besonders problematisch ist dieser Effekt in Kombination mit einer großen Monombasis, da die Kerndimension mit wachsender Anzahl von Monomen bei festem Stichprobenumfang typischerweise zunimmt. Der Solver muss in diesem Fall zwischen einer Vielzahl nahezu äquivalenter sparsity-Muster unterscheiden, was die Rechenzeit erheblich erhöhen kann. Diese Beobachtung zeigt auch, dass eine größere Anzahl an günstig gelegenen Stichproben das Problem vereinfacht. Dies passiert dadurch, dass mit neuen günstig gelegenen Stichproben die Kerndimension niedriger wird und der Lösungsraum unseres MILP weiter eingegrenzt wird.

4.4.4 Branching ignoriert Korrelation von Monomen

Ein weiterer Faktor, der die Laufzeit des Branch-and-Bound Verfahrens beeinflussen kann, sind starke Korrelationen zwischen den Spalten der Evaluationsmatrix. Im Branching-Schritt wird eine einzelne Binärvariable z_j auf null oder eins fixiert. Dadurch wird festgelegt, ob das zugehörige Monom in der gesuchten Polynomgleichung verwendet werden kann. Diese Entscheidung berücksichtigt jedoch zunächst nicht, dass unterschiedliche Monome auf den gegebenen Stichproben sehr ähnliche Auswertungen besitzen können. Sind mehrere Spalten der Evaluationsmatrix stark korreliert, kann der Ausschluss eines einzelnen Monoms häufig durch die Verwendung eines anderen Monoms ausgeglichen werden. Die Fixierung einer einzelnen Binärvariablen kann den zulässigen Bereich dann nur geringfügig einschränken. Dieses Problem tritt insbesondere bei symmetrischen Daten auf. Auch in der LP-Relaxation können in diesem Fall viele ähnliche Lösungen mit fraktionierten Werten der Binärvariablen auftreten. Dadurch entstehen im Branch-and-Bound-Baum mehrere Teilprobleme mit vergleichbaren Schranken. Der Solver muss folglich viele dieser Teilprobleme untersuchen, bevor größere Teile des Lösungsraums ausgeschlossen werden können. Solche Symmetrien können einen erheblichen Einfluss auf die Lösungszeit für einen Branch-and-Bound-Solver haben [30]. Starke Korrelationen zwischen den Spalten der Evaluationsmatrix können somit die Wirksamkeit einzelner Branching-Entscheidungen verringern und die Laufzeit des Verfahrens erhöhen.

4.4.5 Einflussfaktor Rauschen

Rauschen in den Eingabedaten ist ein weiterer wichtiger Faktor, der erheblichen Einfluss auf die Branch-and-Bound Performance und auf das Bestimmen der zugrundeliegenden Varietätengleichung hat. Eine erhebliche Rolle spielt unter anderem die Konditionszahl der Evaluationsmatrix.

Definition 4.20. Für eine Matrix $A \in \mathbb{R}^{n \times n}$ mit den Singulärwerten $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$ ist die Konditionszahl definiert als

$$\kappa(A) := \frac{\sigma_1}{\sigma_n}.$$

Die Konditionszahl misst anschaulich betrachtet, wie stark ein Rauschen in den Eingabedaten von einer Matrix eine Lösung verzerrt [13]. Eine kleine Konditionszahl $\kappa(A) = 1$ sagt dabei aus, dass ein Rauschen in den Eingabedaten von der Matrix nicht verstärkt wird.

Man spricht davon, dass die Matrix gut konditioniert ist. Eine große Konditionszahl bedeutet, dass schon ein leichtes Rauschen in den Eingabedaten zu einer großen Verzerrung führt. Die Matrix ist damit schlecht konditioniert. Besonders hochgradige Evaluationsmatrizen gelten im Allgemeinen als besonders schlecht konditionierte Matrizen, da sie häufig (beinahe) lineare Abhängigkeiten enthalten [23]. Damit ist unser Optimierungsproblem vor allem bei einem großen Maximalgrad der Monome D numerisch instabil. Ein leichtes Rauschen in den Eingabedaten führt unter Umständen zu einer großen Änderung in der Lösung. Dies rechtfertigt zusätzlich zu dem Punkt 4.4.2 die Konzentration auf niedriggradige Polynomgleichungen in dieser Arbeit. Auch die Branch-and-Bound Laufzeit wird durch Rauschen verschlechtert. Dies liegt daran, dass durch das Rauschen häufig keine exakt sparsesten Lösungen mehr existieren, sondern lediglich viele nahezu gleich gute Approximationen. Der Solver muss daher zwischen einer Vielzahl von Lösungen unterscheiden, die ähnlichen Support besitzen. Dies verzögert effektives Pruning und führt zu tiefen und stark verzweigten Suchbäumen.

4.5 Komplexitätsreduzierung durch Presolving

Wir haben in den zwei vorangegangenen Abschnitten gesehen, dass es durchaus möglich ist, dass die Lösungszeit für unser Optimierungsproblem nichtpolynomiell ist, falls $P \neq NP$. Daher ist es sinnvoll, die Komplexität unseres Problems bereits vor der eigentlichen Optimierung zu reduzieren, indem wir die Anzahl ganzzahliger Variablen reduzieren und den Wertebereich der Variablen einschränken. Solche Vorverarbeitungsschritte werden Presolving genannt [28]. Im Folgenden erarbeiten wir uns einige Strategien, mit deren Hilfe wir die Komplexität unseres Problems schon vor Beginn der eigentlichen Optimierung reduzieren können. Zum Abschluss des Kapitels formulieren wir den finalen Lösungsalgorithmus für Problem 3.1 als Pseudoalgorithmus.

4.5.1 ℓ_1 -Lösung als Warm-Start nutzen

Wir haben in Abschnitt 4.1 gesehen, dass der Rang der Evaluationsmatrix ein wichtiger Indikator zur Identifikation der Größe des Lösungsraums ist. In Satz 4.6 haben wir gelernt, dass das Optimierungsproblem 3.1 für Hyperflächen eine eindeutige Lösung hat, wenn die Bedingung

$$\dim(\ker(A)) = 1$$

erfüllt ist. Bei einer Kerndimension von 0 handelt es sich um Daten ohne lineare Abhängigkeiten. In diesem Fall kann es sinnvoll sein den Toleranzparameter in der Singulärwertzerlegung zu erhöhen. Es könnte aber auch sein, dass es keine polynomielle Gleichung gibt, die die Nullstellenmenge der Varietät modelliert, da die Daten nicht von einer Varietät stammen. Ist bekannt, dass die Daten die wir nutzen rauschfrei sind, so können wir die Kerndimension mithilfe des Gauss-Algorithmus oder der QR-Zerlegung untersuchen [15]. Im Fall von verrauschten Daten können wir die Kerndimension, wie in Abschnitt 4.1 erläutert, mithilfe der Singulärwertzerlegung bestimmen. Ist die Kerndimension von 1 erfüllt und wir suchen nach einer Hyperfläche, so können wir die deutlich leichter lösbare ℓ_1 -Formulierung mit Problem 3.4 verwenden. Auch dieses Problem wird die eindeutige Polynomgleichung finden, dessen Nullstellenmenge die Hyperfläche modelliert. Doch auch im allgemeinen Fall mit einer Kerndimension $\dim(\ker(A)) > 1$ oder einer allgemeinen Varietät ist es sinnvoll Problem 3.4 zu lösen und die gefundene Lösung als Startlösung für Problem 3.1 zu nutzen. Dies liegt daran, dass das Problem 3.4 formuliert als LP im Vergleich zu Problem 3.1 günstig lösbar ist. Zusätzlich erhalten wir die, durch die ℓ_1 -Optimierung gefundene sparsity, also die Anzahl der effektiv aktiven Koeffizienten des Vektors c . Diesen Wert $s_{\max}$ können wir durch das Hinzufügen der Nebenbedingung

$$\sum_{j=1}^M z_j \leq s_{\max}$$

in Problem (3.1) als Maximalwert an unsere sparsity-Bedingung verwenden. Somit verkleinern wir den Lösungsraum, was die Branch-and-Bound Performance verbessern kann.

4.5.2 Kernbasierte Monomselektion

In Abschnitt 4.4 haben wir angesprochen, dass das Branching die Varietätenstruktur nicht adäquat abbildet und die Monomanzahl einen negativen Einfluss auf die Branch-and-Bound-Laufzeit hat. Dadurch werden oft unnötig viele Knoten im Branch-and-Bound Baum gebildet. Um dieses Problem zu adressieren, können wir im Presolving schon Monome ausschließen, die nicht im Kern der Evaluationsmatrix vorkommen. Dies liegt daran, dass unser Optimierungsproblem eine besonders sparse Polynomgleichung anhand der Kernbasis bildet. Monome, die nicht in den Kernbasisvektoren vorkommen, können somit nicht Teil einer Lösung unseres Optimierungsproblems sein. Die Presolving-Strategie besteht also daraus, eine Orthonormalbasis des Kerns der Evaluationsmatrix zu bestimmen. Die Kernbasis besteht aus Vektoren $c \in \ker(A)$, deren Einträge den Koeffizienten der zugehörigen Monome entsprechen. Der in Abschnitt 4.2 definierte Wert ρ_j misst dabei,

ob und wie stark die j -te Koeffizientenkoordinate innerhalb des Kerns auftreten kann. Für $\rho_j = 0$ verschwindet der Koeffizient c_j für alle Vektoren $c \in \ker(A)$. Das zugehörige Monom m_j kann daher in keiner zulässigen Polynomgleichung aus dem betrachteten Kern mit einem nichtverschwindenden Koeffizienten auftreten. In der numerischen Umsetzung entfernen wir m_j , wenn

$$\rho_j \leq \tau$$

für eine vorgegebene numerische Toleranz $\tau \geq 0$ gilt. Da ρ_j mithilfe einer Orthonormalbasis definiert ist, hängt diese Entscheidung nicht von der konkreten Wahl der Kernbasis ab. Im rauschfreien Fall ergibt sich das folgende Resultat.

Satz 4.21. *Sei $A \in \mathbb{R}^{N \times M}$ die Evaluationsmatrix und $j \in \{1, \dots, M\}$. Gilt für alle $c \in \ker(A)$ die Bedingung $c_j = 0$, so existiert keine nichttriviale Lösung des Optimierungsproblems (3.1) bzw. Problem (3.6), in der $c_j \neq 0$ gilt.*

Beweis. Für alle Lösungen der beiden Optimierungsprobleme gilt im rauschfreien Fall $Ac = 0$, das heißt nach Definition 4.2 $c \in \ker(A)$. Nach Voraussetzung gilt jedoch für alle Vektoren im Kern $c_j = 0$. Folglich kann kein zulässiges Polynom in der betrachteten Modellklasse das Monom m_j mit nicht verschwindenden Koeffizienten enthalten. $\square$

Die kernbasierte Monomselektion ist im Fall von Rauschen ein heuristischer Presolving-Schritt. Bei verrauschten Daten hängt das Ergebnis sensitiv von der Wahl der Toleranz τ ab. Zu große τ -Werte können relevante Monome fälschlicherweise entfernen und damit zulässige Modelle ausschließen. Zu kleine Werte von τ reduzieren den Komplexitätsgewinn.

4.5.3 Verfeinerung der Big-M-Bedingung

Wie wir im Abschnitt 4.4 bereits angesprochen haben, sind enge Big-M-Schranken entscheidend für eine gute Branch-and-Bound Laufzeit. Dazu leiten wir im Folgenden eine Möglichkeit zum Tightening dieser Schranken her. Anstelle einer globalen M_{big} -Bedingung für das ganze System führen wir einzelne untere Schranken $M_{\text{big}}^{j,\ell}$ und obere Schranken $M_{\text{big}}^{j,\mathcal{U}}$ für jedes Monom ein. Damit verändert sich der erste Satz Nebenbedingungen in 3.1 zu

$$M_{\text{big}}^{j,\ell} z_j \leq c_j \leq M_{\text{big}}^{j,\mathcal{U}} z_j, \quad j = 1, \dots, M.$$

Für diese Big-M-Schranken können wir, ähnlich wie in [2] einfachere (in unserem Fall lineare) Programme lösen, um enge Big-M-Schranken zu erhalten und die Laufzeit des Branch-and-Bound-Solvers zu verbessern. Eine einfache Idee wäre die Extremwerte der einzelnen Koeffizienten auf den Daten inklusive eines Puffers ε zu bestimmen. Diese Extremwerte lassen sich durch Lösen des linearen Optimierungsproblems

$$\begin{aligned}
& \min \quad c_j \\
& \text{s.t.} \quad -\varepsilon_i \leq \sum_{j=1}^M c_j m_j(x^{(i)}) \leq \varepsilon_i, \quad i = 1, \dots, N, \\
& \quad \sum_{i=1}^N \varepsilon_i \leq \varepsilon \\
& \quad b^\top c = 1, \\
& \quad -M_{\text{big}} \leq c_j \leq M_{\text{big}}, \quad j = 1, \dots, M \\
& \quad \varepsilon_i \in \mathbb{R}_+, \quad i = 1, \dots, N,
\end{aligned} \tag{4.7}$$

bzw.

$$\begin{aligned}
& \max \quad c_j \\
& \text{s.t.} \quad -\varepsilon_i \leq \sum_{j=1}^M c_j m_j(x^{(i)}) \leq \varepsilon_i, \quad i = 1, \dots, N, \\
& \quad \sum_{i=1}^N \varepsilon_i \leq \varepsilon \\
& \quad b^\top c = 1, \\
& \quad -M_{\text{big}} \leq c_j \leq M_{\text{big}}, \quad j = 1, \dots, M \\
& \quad \varepsilon_i \in \mathbb{R}_+, \quad i = 1, \dots, N,
\end{aligned} \tag{4.8}$$

berechnen. Seien $\hat{c}_{j,\min}$ und $\hat{c}_{j,\max}$ die Zielfunktionswerte der beiden linearen Programme. Wenn bekannt ist, dass die Daten exakt sind, können diese Schranken mit $\varepsilon = 0$ direkt verwendet werden, wir setzen $M_{\text{big}}^{j,\ell} = \hat{c}_{j,\min}$ und $M_{\text{big}}^{j,\mathcal{M}} = \hat{c}_{j,\max}$. Liegen die Daten nicht exakt auf einer Varietät, verwenden wir das ε aus Problem 3.1. Mit dieser Presolve-Heuristik erhalten wir datenbasierte Schranken, die das in Abschnitt 4.4.1 beschriebene Problem beheben und die Laufzeit des Branch-and-Bound-Solvers deutlich verbessern.

Bemerkung 4.22. Dieser Teilabschnitt begründet auch die konzeptionelle Entscheidung, die wir in Abschnitt 3 getroffen haben. Dort haben wir uns dazu entschieden die Fehlertoleranz ε als globale Schranke in die Nebenbedingung aufzunehmen, statt lokale Schranken zu verwenden, die als Strafterm in die Zielfunktion aufgenommen werden. Dies hätte nämlich dazu geführt, dass ε für die Optimierungsprobleme 4.7 und 4.8 nicht bekannt wäre, wodurch das Big-M-Tightening auf diese Art und Weise nicht möglich wäre.

4.5.4 Algorithmus inklusive Presolving

Nachdem wir nun einige Presolving Verfahren für unser Problem kennengelernt haben, formulieren wir den finalen Algorithmus zur Lösung des Optimierungsproblems 3.6.

Algorithm 1 Finden sparser Nullstellenmengen von Varietäten mit Presolving

- 1: **Eingabe:** Daten $X = \{x^{(1)}, \dots, x^{(N)}\}$, Monome $(m_1, \dots, m_M)$, Fehlertoleranz ε , SVD-Toleranz σ_{tol} , Toleranz für kernbasierte Monomselektion τ , Big-M-Schranke M_{big}
- 2: **Ausgabe:** Koeffizientenvektoren $\mathcal{C} = \{c^{(1)}, \dots, c^{(r)}\}$ linear unabhängiger, sparser Polynomgleichungen
- 3: Stelle Evaluationsmatrix $A \in \mathbb{R}^{N \times M}$ mit $A_{ij} = m_j(x^{(i)})$ auf.
- 4: Führe SVD auf A durch: $A = U\Sigma V^T$
- 5: Bestimme den numerischen Rang durch

$$r_A = \#\{i : \sigma_i > \sigma_{\text{tol}}\sigma_1\}.$$
- 6: Bestimme Orthonormalbasis $N \in \mathbb{R}^{M \times t}$, $t = M - r_A$ des numerischen Kerns von A .
- 7: **if** $t=0$ **then**
- 8: **return** $\emptyset$
- 9: **end if**
- 10: **for** $j = 1, \dots, M$ **do**
- 11: Berechne $\rho_j = \|N_{j,:}\|_2$.
- 12: **end for**
- 13: Entferne alle Monome m_j , für die $\rho_j \leq \tau$ gilt und entferne die zugehörigen Spalten aus A . Wir bezeichnen die Anzahl der verbleibenden Monome mit M_{red} .
- 14: Berechne den numerischen Kern der reduzierten Evaluationsmatrix erneut und bezeichne die resultierende Orthonormalbasis wieder mit N .

15: Berechne für jedes restliche Monom erneut $\rho_j = \|N_{j,:}\|_2$ und sortiere die Monomindizes so, dass

$$\rho_{q_1} \geq \rho_{q_2} \geq \dots \rho_{q_{M_{\text{red}}}}.$$

16: Initialisiere $\mathcal{C} \leftarrow \emptyset, P \leftarrow \emptyset$ und $F \leftarrow \emptyset$. Die Menge P sind die bereits verwendeten Pivots und F die Pivots, die unzulässige Probleme verursachen.

17: Setze $t \leftarrow \dim(\ker(A))$.

18: **for** $k = 1, \dots, t$ **do**

19: Lösung gefunden $\leftarrow$ false

20: **for** $j = 1, \dots, M_{\text{red}}$ **do**

21: Setze Pivotkoeffizient $p \leftarrow q_j$.

22: **if** $p \in P$ oder $p \in F$ **then**

23: **continue**

24: **end if**

25: Formuliere und löse die ℓ_1 -Relaxation von Problem 3.6 und verwende die Supportgröße als obere Schranke.

26: Bestimme für die Normierung $c_p = 1$ untere und obere Schranken durch die LPs wie in Abschnitt 4.5.3 erklärt und ersetze die Big-M-Bedingungen.

27: Formuliere und löse das Optimierungsproblem 3.6 mit der ℓ_1 Supportschranke, den verfeinerten Big-M-Schranken und $c_p = 1$.

28: **if** das Problem ist zulässig und besitzt Optimallösung $c^{(k)}$ **then**

29: $\mathcal{C} \leftarrow \mathcal{C} \cup \{c^{(k)}\}, \quad P \leftarrow P \cup \{p\}$.

30: Lösung gefunden $\leftarrow$ true

31: **break**

32: **else**

33: $F \leftarrow F \cup \{p\}$

34: **end if**

35: **end for**

36: **if** Lösung gefunden=false **then**

37: **break**

38: **end if**

39: **end for**

40: **return** $\mathcal{C}$

Zum Abschluss dieses Kapitels erläutern wir noch einige Aspekte zu diesem Algorithmus. Für die Berechnung des (numerischen) Nullraums aus der Singulärwertzerlegung sei auf Theorem 5.2 in [33] verwiesen. Bei der Ermittlung des numerischen Nullraums mithilfe

von σ_{tol} haben wir uns für eine relative Schranke entschieden. Koeffizienten werden dem Kern zugeordnet, wenn sie verglichen mit dem größten Singulärwert klein sind.

Wir haben den Algorithmus an dieser Stelle ausschließlich für die Pivot-Methode formuliert. Für Hyperflächen ändert sich im Wesentlichen, dass die Schleife über t entfernt wird, da wir davon ausgehen nur eine Lösung bestimmen zu können. Wir verwenden die Pivotberechnung dafür, die Wahl des Vektors b zu rechtfertigen. Wir wählen $b = e_p$ für den Pivotkoeffizienten p . Für die No-Good-Cut-Variante wählen wir ebenso $b = e_p$, führen allerdings keine Liste von erfolgreichen Pivots. Wir verwenden also ein Pivotelement so lange, bis wir keine Lösung mehr mit diesem Pivotelement finden. Weiterhin wird die Zeile **for** $k = 1, \dots, t$ **do** ersetzt durch **while** $\text{rank}(\mathcal{C}) < t$ **do**, da wir keine lineare Unabhängigkeit garantieren können. Wir führen das No-Good-Cut-Verfahren also so lange durch, bis wir $\dim(\ker(A))$ viele linear unabhängige Gleichungen gefunden haben.

5 Erweiterung des Modells auf symbolische Regression

Motiviert durch die Arbeit [34], könnte unser bisher erarbeiteter Algorithmus auf Probleme aus der Klasse der symbolischen Regression angewandt werden. In diesem Abschnitt betrachten wir deshalb zunächst diese Problemklasse, um dann unser Problem für diese Problemklasse zu verallgemeinern. Daraufhin diskutieren wir, ob unsere vorher hergeleiteten Resultate auch für diese Problemklasse weiter gelten und leiten spezielle Presolving-Maßnahmen für diese Problemklasse her.

Die Ausgangssituation bei der symbolischen Regression ist strukturell verwandt mit unserem Problem für Hyperflächen. Wir haben erneut Datenpunkte

$$X = \{x^{(1)}, \dots, x^{(N)}\} \subset \mathbb{R}^d, \quad d \in \mathbb{N}.$$

und diesmal einen zugehörigen Funktionswert $y^{(1)}, \dots, y^{(N)} \in \mathbb{R}$. Ziel ist es, eine Abbildung $f : \mathbb{R}^d \mapsto \mathbb{R}$ zu finden, die für die Punkte $x^{(i)}$ die Bedingung

$$f(x^{(i)}) \approx y^{(i)} \tag{5.1}$$

erfüllt. Der Unterschied bei der symbolischen Regression ist, dass die Annahme nicht ist, dass die Datenpunkte durch eine Polynomgleichung, sprich eine additive Verknüpfung von Monomen beschrieben werden. Stattdessen ist die Modellstruktur bei der symbolischen Regression komplett unbekannt. Symbolische Regression behandelt damit zwei Probleme gleichzeitig [34]. Das eine Teilproblem ist das Auffinden von passenden Kandidatenfunktionen. Diese Kandidatenfunktionen entsprechen unserer bisherigen Monombasis. Das zweite Teilproblem ist das Fitten der Stichprobenpunkte auf diese Kandidaten, um die additive- und/oder multiplikative Verknüpfung dieser Kandidaten als finale Funktion, für die die Bedingung (5.1) erfüllt ist. Da wir bisher die Monombasis als gegeben hingenommen haben, setzen wir den Fokus in dieser Arbeit auf das zweite Teilproblem. Auf die

Erstellung einer Kandidatenbasis für unsere folgenden numerischen Experimente gehen wir im weiteren Verlauf dieses Kapitels ein.

Wir nehmen also zunächst an, es gibt ein Orakel das uns eine Basis aus passenden Kandidatenfunktionen $\phi_1, \dots, \phi_M : \mathbb{R}^d \mapsto \mathbb{R}$ zusammenstellt. Wir nehmen an, mit dieser Basis könnten alle Funktionen bestimmt werden. Die Annahme ist also, f aus Bedingung (5.1) lässt sich schreiben als

$$f(x) = \sum_{j=1}^M c_j \phi_j(x).$$

Problem 5.1. *Damit können wir ein Optimierungsproblem analog zu Problem 3.1 aufstellen:*

$$\begin{aligned} \min \quad & \sum_{j=1}^M z_j \\ \text{u.d.N.} \quad & -M_{\text{big}} z_j \leq c_j \leq M_{\text{big}} z_j, \quad j = 1, \dots, M, \\ & -\varepsilon_i \leq \sum_{j=1}^M c_j \phi_j(x^{(i)}) - y_i \leq \varepsilon_i, \quad i = 1, \dots, N, \\ & \sum_{i=1}^N \varepsilon_i \leq \varepsilon, \\ & z_j \in \{0, 1\}, \quad j = 1, \dots, M, \\ & c_j \in \mathbb{R}, \quad j = 1, \dots, M, \\ & \varepsilon_i \in \mathbb{R}_+. \end{aligned} \tag{5.2}$$

Auch die Evaluationsmatrix lässt sich verallgemeinern. Die verallgemeinerte Evaluationsmatrix hat dabei die Struktur

$$A = \begin{pmatrix} \phi_1(x^{(1)}) & \phi_2(x^{(1)}) & \cdots & \phi_M(x^{(1)}) \\ \phi_1(x^{(2)}) & \phi_2(x^{(2)}) & \cdots & \phi_M(x^{(2)}) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_1(x^{(N)}) & \phi_2(x^{(N)}) & \cdots & \phi_M(x^{(N)}) \end{pmatrix}.$$

Ein wesentlicher Unterschied zum Varietätenproblem besteht darin, dass die rechte Seite nicht mehr null ist. Im Varietätenfall wurde ein Koeffizientenvektor c mit

$$Ac = 0$$

gesucht. Dadurch war die triviale Lösung $c = 0$ stets zulässig und musste durch eine Normierungsbedingung der Form

$$b^\top c = 1$$

ausgeschlossen werden. Im Regressionsfall liegt dagegen das inhomogene Problem

$$Ac \approx y$$

vor. Sofern y nicht identisch null ist, ist die Nulllösung im Allgemeinen keine gute Approximation. Daher ist eine Skalierungsfixierung wie im Varietätenfall nicht notwendig.

5.1 Konstruktion der Kandidatenbibliothek

Die Qualität des Ansatzes hängt wesentlich von der gewählten Kandidatenbibliothek ab. In den vorherigen Kapiteln war diese Bibliothek durch alle Monome bis zu einem Maximalgrad D gegeben. Für symbolische Regression ist diese Einschränkung zu eng, da viele relevante Zusammenhänge nicht-polynomiell sind. Daher wird die Bibliothek um elementare Funktionen und Kombinationen solcher Funktionen erweitert.

In dieser Arbeit beginnen wir mit einer Menge polynomialer Grundterme $\mu_1, \dots, \mu_M$ mit

$$\mu_i = x_1^{\alpha_1} \cdots x_d^{\alpha_d}, \quad \alpha_1 + \cdots + \alpha_d \leq D.$$

Wir ermöglichen auf diese Grundterme algebraische Operatoren wie

$$\sin, \quad \cos, \quad \text{inv},$$

wobei $\text{inv} = \cdot^{-1}$, anzuwenden. Dadurch entstehen Features wie

$$\phi_a = \sin(x_1^{\alpha_1} \cdots x_d^{\alpha_d}) \text{ und } \cos(x_1^{\alpha_1} \cdots x_d^{\alpha_d}), \dots$$

Zusätzlich können aus bereits erzeugten Features durch Multiplikation mit Grundtermen neue Argumente gebildet werden, wie

$$\mu_a \phi_b.$$

Wir lassen die Multiplikation von zwei Features ϕ nicht zu, da dies die gesamte Anzahl an Features deutlich anwachsen ließe und wir mit Hinblick auf die Lösungszeit unseres MILP auf eine überschaubare Feature-Bibliothek angewiesen sind. Die algebraischen Operationen plus und minus werden für elementare Features nicht benötigt, da diese von unserem Optimierungsproblem aus den erzeugten Features gelernt werden können. Auf zusammengesetzte Features können wiederum die elementaren Features angewandt werden. Damit entstehen Kandidatenfunktionen der Form

$$\text{inv}(\sin((x_1^{\alpha_1} \cdots x_d^{\alpha_d})), \quad \cos(\text{inv}(x_1)), \dots$$

Dies könnte in beliebiger Tiefe weitergeführt werden, allerdings wächst mit zunehmender Tiefe die Kandidatenbasis und somit die Anzahl der Binärvariablen in unserem Optimierungsproblem stark an. Wir erhalten somit einen Konflikt bezüglich der Größe der Kandidatenbasis. Zum einen benötigen wir eine Kandidatenbasis, die die gesuchte Funktion erzeugen kann, zum anderen darf die Kandidatenbasis aber nicht zu groß werden, da wir in Abschnitt 4.4 gesehen haben, dass unser MILP dann nicht mehr in akzeptabler Zeit lösbar ist. Wir sehen in den numerischen Experimenten, dass die Größe der Kandidatenbibliothek ein wesentlicher Faktor für die Lösungszeit ist.

Bemerkung 5.2. *Mit unserem Ansatz erzeugen wir eine Abhängigkeit von der Kandidatenbibliothek, da wir nur Gleichungen lernen können, die mit unserer Kandidatenbibliothek darstellbar sind. Deshalb lösen wir nicht das Problem der symbolischen Regression, wie es in gängiger Literatur bekannt ist. Dies liegt daran, dass bei der symbolischen Regression die Modellstruktur im Vorhinein komplett unbekannt ist. Bei unserem Ansatz hingegen geben wir eine gewisse Struktur durch die Kandidatenbibliothek vor. Da wir jedoch unseren Ansatz auf die Gleichungen aus [34] anwenden, haben wir uns jedoch dazu entschieden die gleiche Terminologie zu verwenden.*

5.2 Komplexität der Erweiterung

In diesem Kapitel betrachten wir die theoretischen Grundlagen aus Kapitel 4 um zu bewerten, wie sich die Komplexität durch die Erweiterung auf symbolische Regression verändert. In Abschnitt 4.1 haben wir gelernt, dass die Existenz und Eindeutigkeit im Varietätenfall von der Kerndimension abhängt. Da wir nicht c mit $Ac = 0$ sondern mit $Ac = y$ suchen, lässt sich diese Aussage nicht exakt auf diesen Fall übertragen. Vielmehr folgt aus linearer Algebra [15], dass unser Optimierungsproblem im rauschfreien Fall

eine Lösung besitzt, wenn $y \in \text{span}\{A_{:,1}, \dots, A_{:,M}\}$. Weiterhin ist die Lösung eindeutig, falls der Kern trivial ist, also $\text{rank}(A) = M$. Da wir eine bedeutend größere Kandidatenbibliothek haben, benötigen wir für Eindeutigkeit der Lösung auch eine bedeutend größere Stichprobenanzahl. Die NP-Schwere des Entscheidungsproblems zu unserem Optimierungsproblems bleibt auch im Fall der symbolischen Regression erhalten, da die NP-Schwere aus der sparsity-Bedingung kommt.

In Abschnitt 4.4 haben wir die Einflussfaktoren auf das Branch-and-Bound Verfahren analysiert. Auf Basis dieser Analyse fällt auf, dass wir nun deutlich mehr Features haben. Während die Monomanzahl in unserem Varietätenproblem durch die Formel aus Satz 3.5 begrenzt ist, kann die Featurezahl bei unserem symbolischen Regressionsproblem je nach Anzahl elementarer Funktionen und Maximalgrad der Monome D sehr groß werden. Zusätzlich haben allgemeine Funktionen häufig Definitionslücken oder sehr große Funktionswerte, deshalb können sie in der verwendeten Kandidatenbibliothek zu numerischen Problemen führen. Beispielsweise ist $\frac{1}{x}$ numerisch problematisch, wenn x nahe null liegt oder sehr große positive Werte annimmt. Daher beschränken wir bei der Funktion $\frac{1}{x}$ den Nenner von Null weg. Dazu nutzen wir für den Nenner die Funktion

$$\text{safeinv}(x) = \begin{cases} x, & |x| \geq 10^{-6}, \\ 10^{-6}, & 0 \leq x < 10^{-6}, \\ -10^{-6}, & -10^{-6} < x < 0. \end{cases}$$

Statt $\frac{1}{x}$ nutzen wir in unserer Kandidatenbibliothek $\frac{1}{\text{safeinv}(x)}$. Diese Maßnahme verändert zwar die Kandidatenfunktionen, ist allerdings notwendig, um die numerische Stabilität zu erhöhen und die Qualität der LP-Relaxationen und Big-M-Schranken zu verbessern.

5.2.1 Presolving-Maßnahmen

In Abschnitt 4.5 haben wir Presolving-Maßnahmen kennengelernt, um die Komplexität des Optimierungsproblems zu verringern. Der Warmstart mit der ℓ_1 -Lösung ist ebenso wie die Verfeinerung der Big-M-Bedingung auch auf dieses Problem anwendbar. Da die Lösung des Optimierungsproblems 5.1 nicht aus dem Kern von A stammt, ist die kernbasierte Monomselektion nicht ohne Weiteres auf dieses Problem anwendbar. Diese Presolving-Maßnahme ist übertragbar, indem statt A die augmentierte Matrix $\tilde{A} = (A \quad -y)$ und der augmentierte Koeffizientenvektor

$$\tilde{c} = \begin{pmatrix} c \\ 1 \end{pmatrix},$$

betrachtet wird. Dann gilt $\tilde{A}\tilde{c} = Ac - y$ und im rauschfreien Fall gilt

$$Ac = y \iff \tilde{c} \in \ker(\tilde{A}) \text{ und } \tilde{c}_{M+1} = 1.$$

Die Ausgabe y wird also formal wie ein zusätzlicher Kandidat in der Kandidatenbibliothek aufgefasst, dessen Koeffizient auf 1 fixiert wird. Jedoch ist die kernbasierte Monomselektion für das Beispiel symbolischer Regression nicht unbedingt hilfreich, da wir sonst eine sehr große Stichprobe benötigen würden, da die kernbasierte Monomselektion besonders dann hilfreich ist, wenn der Lösungsraum nicht sehr groß ist. Dazu ist notwendig, dass die Stichprobengröße nicht deutlich kleiner als die Kandidatenbibliothek ist und somit die Kerndimension der augmentierten Matrix nicht sehr groß ist.

Stattdessen nutzen wir einfache Presolving-Maßnahmen, um nach der Konstruktion der Kandidatenbibliothek unnötige Features zu entfernen. Der Grund dafür ist, dass wir bei der automatischen Konstruktion syntaktisch unterschiedliche, aber mathematisch äquivalente oder numerisch redundante Features erzeugen, wie zum Beispiel

$$xy = yx.$$

Die Reduktion verläuft in den folgenden Schritten:

- Entfernen von Features, deren Auswertung auf allen Datenpunkten numerisch null ist.
- Entfernen redundanter konstanter Features, wobei höchstens ein konstantes Feature behalten wird.
- Entfernen proportionaler Features: Falls für zwei Spalten $\phi_{j_1}(x^{(i)}) = \alpha \phi_{j_2}(x^{(i)})$ für alle $i = 1, \dots, N$ gilt, wird nur eines der beiden Features behalten.

Mit diesen Presolving-Schritten reduzieren wir die Anzahl der Kandidatenfunktionen und somit auch die Anzahl der Binärvariablen in unserem Optimierungsproblem.

Zusätzlich enthält der Feynman-Datensatz, an dem wir unser Problem im Folgenden Kapitel testen, viele physikalische Gleichungen, die durch Skalierung eines einzelnen Features unserer Bibliothek erzeugt werden können. Deshalb fügen wir unserem Lösungsalgorithmus einen weiteren Presolving Schritt hinzu, um zu identifizieren, ob eine Lösung mit einem einzelnen Feature zulässig ist. Dazu berechnen wir für jedes Feature

$$E_j = \min_{c_j \in [-M_{\text{big}}, M_{\text{big}}]} \sum_{i=1}^N |\phi_{ij}c_j - y_i|,$$

den minimalen ℓ_1 -Fehler innerhalb der vorgegebenen Schranken. Daraufhin berechnen wir das Minimum $E_{\min}$ der E_j für $j \in \{1, \dots, M\}$. Ist dieser Wert kleiner als die vorgegebene Fehlertoleranz ε , existiert ein einzelnes Feature, das innerhalb der Fehlertoleranz liegt. Da wir ausschließlich die sparsity minimieren ist der zugehörige Kandidat optimal für unser Optimierungsproblem 5.1. Wir erhalten den zugehörigen Kandidaten dann zweistufig. Zunächst berechnen wir

$$c_j^* \in \operatorname{argmin}_{c \in [-M_{\text{big}}, M_{\text{big}}]} \sum_{i=1}^N |\phi_{ij}c - y_i|, \quad \text{für } j = 1, \dots, M. \quad (5.3)$$

Daraufhin berechnen wir

$$j^* \in \operatorname{argmin}_{j=1, \dots, M} \sum_{i=1}^N |\phi_{ij}c_j^* - y_i|.$$

Der erhaltene Kandidat c_{j^*} ist der optimale Kandidat, sofern die Nulllösung nicht zulässig ist. Ist der Wert $E_{\min}$ größer als die vorgegebene Fehlertoleranz ε , existiert kein einzelnes Feature, das die Fehlertoleranz einhält. Die Fehlertoleranz kann nur durch die Kombination mehrerer Features eingehalten werden. Wir können dem Optimierungsproblem 5.1 als Nebenbedingung

$$\sum_{j=1}^M z_j \geq 2$$

hinzufügen. Mit dieser Presolving-Maßnahme kann im besten Fall die optimale Lösung ohne MILP berechnet werden. Im schlechtesten Fall können wir einen Teil des Branch-and-Bound Baumes abschneiden, indem wir die minimal mögliche sparsity auf zwei begrenzen.

6 Numerische Experimente

In diesem Kapitel prüfen wir die theoretischen Aussagen, die wir im Laufe dieser Arbeit getroffen haben, empirisch, um so Aussagen über deren Tragfähigkeit treffen zu können. Dies teilen wir in drei Teilabschnitte ein. Zunächst prüfen wir die grundlegenden Modellparameter im Hyperflächenfall. Diese Erkenntnisse nutzen wir für allgemeine Varietäten, um zum Schluss noch einige Experimente zur symbolischen Regression durchzuführen. Alle numerischen Experimente wurden auf einem Rechner mit Intel Core Ultra 7 165H Prozessor mit 32 GB RAM und einem Windows 11 25H2 Betriebssystem durchgeführt. Die Implementierung erfolgt in Python 3.13.5 [26] unter Verwendung von NumPy 2.3.0 [11] zur Datenverarbeitung und PySCIPOpt 5.5.0 [19] als Schnittstelle zum MILP-Solver SCIP [3]. Der vollständige Quellcode steht im zugehörigen GitHub-Repository unter https://github.com/JonathanFoltz1998/Masterarbeit_Jonathan_Foltz zum Download zur Verfügung.

6.1 Hyperflächen

Wir prüfen in diesem Abschnitt die angesprochenen Parameter und Aussagen aus den letzten Abschnitten. Dazu nutzen wir als Beispiel den Einheitskreis, die Lemniskate von Geroni und den Schnitt einer Gerade mit einem Kreis. Im Fall des Einheitskreises erzeugen wir die Daten gleichmäßig im Bereich $t \in [0, 2\pi)$, durch

$$x = \cos(t), \quad y = \sin(t)$$

und im Fall der Lemniskate von Geroni durch

$$x = \cos(t), \quad y = \sin(t) \cos(t).$$

Über alle Experimente hinweg haben wir $M_{\text{big}} = 1000$ gewählt, somit sind die Koeffizienten der Monome durch 1000 nach oben und -1000 nach unten beschränkt. Wir

erzeugen zunächst $N = 50$ Stichproben auf dem Einheitskreis mit einem Rauschen $\sigma \in \{0, 0.001, 0.05, 0.1, 0.15\}$ und einem Maximalgrad von $D = 2$. Wir betrachten dabei additives gaußsches Rauschen im algebraischen Residuum. Dazu erzeugen wir die Stichprobenpunkte $x^{(i)}$ so, dass für das zugrunde liegende Polynom

$$f(x^{(i)}) = \xi_i, \text{ wobei } \xi_i \sim \mathcal{N}(0, \sigma^2)$$

gilt. Die verrauschten Punkte liegen somit nicht exakt auf der Nullstellenmenge $\mathcal{V}(f)$ sondern auf zufällig gestörten Niveaumengen von f . Damit können wir die Fehlertoleranz ε mithilfe einer Monte-Carlo-Simulation bestimmen. Wir nutzen, dass die Nebenbedingungen in Problem (3.1) zusammen mit (3.5) die Bedingung $\|\sum_{j=1}^M c_j m_j(X)\|_1 \leq \varepsilon$ erzwingt. Das für die Zulässigkeit der Gleichung benötigte Gesamtfehlerbudget beträgt also $\sum_{i=1}^N |\xi_i|$. Zum Abschätzen der Fehlertoleranz können wir somit für $\eta \sim \mathcal{N}(0, 1)$ eine Quantilsschätzung mithilfe von Monte-Carlo-Simulation mit 200000 Simulationen durchführen und die gefundene Lösung mit der Rauschstärke σ multiplizieren. Als Konfidenzniveau haben wir 95% genutzt. Wir hätten auch ein höheres Konfidenzniveau wählen können, allerdings erhöht dies auch die Gefahr, dass andere Gleichungen zulässig werden, die auf den Stichproben einen geringen Fehler aufweisen. Für $N = 10$ ergibt sich beispielsweise als 95%-Quantil ungefähr 11.29. Multipliziert mit einer Rauschstärke von $\sigma = 0.1$ ergibt das eine Fehlertoleranz von 1.129.

Die erwartete Gleichung im Fall des Einheitskreises wäre $x^2 + y^2 - 1 = 0$. Im Fall von kontinuierlichen Variablen, kann es vorkommen, dass die Koeffizienten beinahe exakt bestimmt werden. Sei $\hat{c}$ die Koeffizienten der zugrundeliegenden Hyperfläche und c die von Optimierungsproblem 3.1 bestimmte optimale Lösung. Dann legen wir fest, dass die Varietät genau bestimmt wurde, wenn

$$\text{supp}(c) = \text{supp}(\hat{c}) \text{ und } \frac{\|\alpha^* c - \hat{c}\|_2}{\|\hat{c}\|_2} \leq 0.05, \text{ wobei } \alpha^* = \frac{c^\top \hat{c}}{c^\top c}.$$

Dies bedeutet, dass die gleichen Koeffizienten in der erhaltenen Lösung und der korrekten Lösung aktiv sind und die prozentuale skaleninvariante relative Abweichung der Koeffizienten weniger als 5 Prozent beträgt. Die Schranke von 5% wurde als Toleranz gewählt, um im kontinuierlichen Fall Lösungen, die sehr nahe an der korrekten Lösung liegen als korrekt zu klassifizieren. Um unsere Methoden miteinander zu vergleichen, haben wir jeweils mit den gleichen Stichproben für ℓ_0 mit ganzzahligen bzw. kontinuierlichen Koeffizienten und ℓ_1 mit ganzzahligen bzw. kontinuierlichen Koeffizienten das Verfahren laufen lassen. Als Wahl für b haben wir die in Abschnitt 4.2 hergeleitete Pivotwahl verwendet, wobei wir im Fall von nicht korrekter Identifizierung zum nächstbesten Pivotelement übergegangen

Tabelle 6.1: Vergleich der Lösungsqualität und Laufzeit beim Einheitskreis für verschiedene Rauschlevel

Rauschen	Methode	Richtige Lösungen	ε	Zeit in s
0	ℓ_0 ganzzahlig	50	0	0.01
0	ℓ_0 kontinuierlich	50	0	0.01
0	ℓ_1 ganzzahlig	50	0	0.01
0	ℓ_1 kontinuierlich	50	0	0.01
0.001	ℓ_0 ganzzahlig	50	0.0471	0.02
0.001	ℓ_0 kontinuierlich	50	0.0471	0.02
0.001	ℓ_1 ganzzahlig	50	0.0471	0.02
0.001	ℓ_1 kontinuierlich	11	0.0471	0.02
0.05	ℓ_0 ganzzahlig	50	2.3551	0.09
0.05	ℓ_0 kontinuierlich	50	2.3551	0.13
0.05	ℓ_1 ganzzahlig	50	2.3551	0.04
0.05	ℓ_1 kontinuierlich	6	2.3551	0.02
0.1	ℓ_0 ganzzahlig	49	4.7103	0.14
0.1	ℓ_0 kontinuierlich	37	4.7103	0.36
0.1	ℓ_1 ganzzahlig	49	4.7103	0.04
0.1	ℓ_1 kontinuierlich	2	4.7103	0.04
0.15	ℓ_0 ganzzahlig	49	7.0654	0.18
0.15	ℓ_0 kontinuierlich	11	7.0654	0.62
0.15	ℓ_1 ganzzahlig	49	7.0654	0.06
0.15	ℓ_1 kontinuierlich	0	–	–

sind. Dies haben wir für jedes Verfahren und jede Rauschstärke σ 50 mal und jeweils noch ohne Presolving durchgeführt. Die Ergebnisse dafür sind in Tabelle 6.1 abgebildet.

Die Tabelle zeigt, dass im rauschfreien Fall alle vier Varianten die Gleichung des Einheitskreises in allen 50 Durchläufen korrekt rekonstruieren. Unterschiede zeigen sich erst bei verrauschten Daten. Die ganzzahligen Formulierungen bleiben über alle betrachteten Rauschlevel hinweg sehr robust: Sowohl die ℓ_0 - als auch die ℓ_1 -Variante liefern bis einschließlich $\sigma = 0.15$ nahezu durchgehend korrekte Lösungen. Die kontinuierliche ℓ_0 -Variante ist für kleine und mittlere Rauschlevel ebenfalls leistungsfähig, verliert bei stärkerem Rauschen deutlich an Stabilität. Bei $\sigma = 0.15$ werden nur noch 11 von 50 Durchläufen korrekt gelöst. Die kontinuierliche ℓ_1 -Variante zeigt dagegen bereits bei geringem Rauschen einen deutlichen Leistungsabfall. Während sie im rauschfreien Fall zuverlässig ist, findet sie bereits bei sehr kleinem Rauschen von $\sigma = 0.001$ nur noch in 11 von 50 Fällen die richtige Lösung und scheitert für größere Rauschlevel nahezu vollständig. Hier könnte eine zusätzliche Toleranzschwelle für den Support $\text{supp}(c)$ zu besseren Ergebnissen führen.

Mit zunehmendem Rauschen steigt erwartungsgemäß in allen Fällen auch die Fehlertoleranz ε , die zur Rekonstruktion der korrekten Gleichung benötigt wird. Ebenso nehmen die Laufzeiten tendenziell zu, bleiben jedoch insgesamt gering. Bemerkenswert ist, dass die ganzzahlige ℓ_1 -Variante nahezu dieselbe Lösungsqualität wie die ganzzahlige ℓ_0 -Variante erreicht, dabei aber für größere Rauschstärken kürzere Laufzeiten aufweist. Für dieses Experiment erscheint sie daher als besonders günstiger Kompromiss zwischen Robustheit und Rechenzeit. Im Fall des Einheitskreises hat für alle Rauschstärken in allen Versuchen im Erfolgsfall jeweils das erste Pivotelement zum Erfolg geführt. Dies zeigt, dass im Fall des Einheitskreises unsere Pivotwahl treffend ist.

Um diese Ergebnisse mit einem komplizierteren Beispiel zu vergleichen, betrachten wir für die gleichen Rauschstärken σ die Lemniskate von Geroni mit Maximalgrad der Monome $D = 4$. Die erwartete Gleichung zur Beschreibung der Hyperfläche wäre in diesem Fall $x^4 - x^2 + y^2 = 0$. Der sonstige Versuchsaufbau ist genauso wie beim Einheitskreis. Die Ergebnisse dieses Versuchs sind Tabelle 6.2 zu entnehmen.

Erneut konnten unter Abwesenheit von Rauschen alle vier Varianten in allen Fällen die korrekte Lösung identifizieren. Auffällig ist, dass schon kleines Rauschen im Vergleich zum Einheitskreis dazu führt, dass alle Varianten Probleme bei der Identifizierung hatten. Problematisch war in diesem Fall häufig die Verwechslung mit der Gleichung $y^4 = 0$, die auf den Stichproben häufig nahezu erfüllt war, da Daten auf der Lemniskate von Geroni in y betragsmäßig sehr klein sind und durch diese Gleichung eine geringere sparsity erzielt werden konnte. Dieses Verhalten verdeutlicht eine grundsätzliche Schwäche rein

Tabelle 6.2: Vergleich der Lösungsqualität und Laufzeit bei der Lemniskate von Gerono für verschiedene Rauschlevel

Rauschen	Methode	Richtige Lösungen	ε	Zeit in s
0	ℓ_0 ganzzahlig	50	0	0.07
0	ℓ_0 kontinuierlich	50	0	0.05
0	ℓ_1 ganzzahlig	50	0	0.07
0	ℓ_1 kontinuierlich	50	0	0.05
0.001	ℓ_0 ganzzahlig	43	0.0471	0.22
0.001	ℓ_0 kontinuierlich	40	0.0471	0.31
0.001	ℓ_1 ganzzahlig	43	0.0471	0.17
0.001	ℓ_1 kontinuierlich	1	0.0471	0.10
0.05	ℓ_0 ganzzahlig	46	2.3551	1.27
0.05	ℓ_0 kontinuierlich	13	2.3551	3.58
0.05	ℓ_1 ganzzahlig	46	2.3551	0.47
0.05	ℓ_1 kontinuierlich	0	–	–
0.1	ℓ_0 ganzzahlig	32	4.7103	0.94
0.1	ℓ_0 kontinuierlich	1	4.7103	5.43
0.1	ℓ_1 ganzzahlig	32	4.7103	0.84
0.1	ℓ_1 kontinuierlich	0	–	–
0.15	ℓ_0 ganzzahlig	14	7.0654	1.07
0.15	ℓ_0 kontinuierlich	0	–	–
0.15	ℓ_1 ganzzahlig	14	7.0654	1.58
0.15	ℓ_1 kontinuierlich	0	–	–

sparsity-basierter Methoden: Bei verrauschten oder ungünstig skalierten Daten können sehr einfache, aber geometrisch falsche Gleichungen gegenüber der tatsächlichen Hyperflächengleichung bevorzugt werden. Für ein Rauschen von 0.15 konnten nur noch die ganzzahligen Varianten in knapp $\frac{1}{4}$ der Fälle die korrekte Lösung identifizieren. Die restlichen Varianten scheitern komplett. Zur Verbesserung der Ergebnisse bei größerem Rauschen wurde statt einer Gesamtfehlertoleranz ε auch eine Fehlertoleranz pro Punkt getestet. Dies hat allerdings keine deutliche Verbesserung gebracht. Bei den Ausführzeiten ist auffällig, dass die Zeit für ein relativ moderates Rauschen von 0.05 schon deutlich größer ist als im Einheitskreisfall bei einem Rauschen von 0.15. Dies ist auch, aber nicht nur durch die höhere Monomanzahl zu erklären. Stattdessen eignete sich die Pivotwahl für die Lemniskate von Geroni deutlich schlechter. Das korrekte Pivotelement wurde häufig erst an fünfter Stelle gewählt. Dadurch mussten deutlich mehr MILPs gelöst werden. Dies ist vor allem dann problematisch, wenn im Gegensatz zu unseren Experimenten die richtige Lösung nicht bekannt ist und die Fehlertoleranz ε überschätzt wird. Dann kann das Problem auftreten, dass Lösungen die innerhalb der Fehlertoleranz liegen, aber schlechtere sparsity aufweisen bevorzugt werden, weil deren Koeffizienten mit dem ersten Pivotelement zulässig sind.

In den bisher betrachteten Beispielen wurde noch nicht deutlich, dass der Nachteil der deutlich höheren Laufzeit der ℓ_0 -Variante aufgewogen wird dadurch, dass mit dieser Variante eine höhere sparsity erzeugt wird. Dies liegt daran, dass die Lösungskoeffizienten hier jeweils Betrag von eins hatten. In diesem Fall optimieren die ℓ_0 - und ℓ_1 -Variante das Gleiche. Deshalb haben wir zusätzlich mit der kontinuierlichen ℓ_0 bzw. ℓ_1 Variante ohne Rauschen zwei Punkte auf der Schnittmenge von $10y - 9x = 0$ und $x^2 + y^2 - 1 = 0$ erzeugt und unser Verfahren mit $D = 2$ getestet. Ergebnis war, dass im ℓ_0 -Fall die Gleichung $10y - 9x = 0$ gefunden wurde, während im ℓ_1 -Fall schlechtere sparsity mit der optimalen Gleichung $x^2 + y^2 - 1 = 0$ erzielt wurde. Dies zeigt, dass auch ohne Rauschen nicht in allen Fällen mit der ℓ_1 -Variante die minimale sparsity von zwei erzielt werden kann. Im ganzzahligen Fall kann die Gleichung $10y - 9x = 0$ aufgrund der Pivotbedingung hingegen überhaupt nicht gefunden werden. Die Begründung hierfür wurde in Bemerkung 3.3 diskutiert.

Als nächstes betrachten wir den Einfluss des Fehlerparameters ε im ganzzahligen und kontinuierlichen ℓ_0 -Fall. Dazu betrachten wir einen Durchlauf unseres vorherigen Experiments am Beispiel des Einheitskreises mit einem Rauschen von 0.15. Doch statt ε a priori durch Monte-Carlo-Simulation zu schätzen, haben wir mit einem Startwert $\varepsilon^{(1)} = \sigma$ gestartet und mit jeder Iteration um σ erhöht. Es gilt also in Iteration $t \in \{1, \dots, 100\}$, dass $\varepsilon^{(t)} = t\sigma$. Zusätzlich haben wir $b^\top = (1, \dots, 1)$ gewählt statt b über die Pivotwahl zu bestimmen, da wir damit eine garantiert zulässige Einheitskreis-Lösung bekommen.

Dadurch können wir uns den Einfluss der Fehlertoleranz auf die Lösung ansehen. Die Ergebnisse sind in Abbildung 6.1 zu sehen. Hierbei wurde die Lösung im ganzzahligen

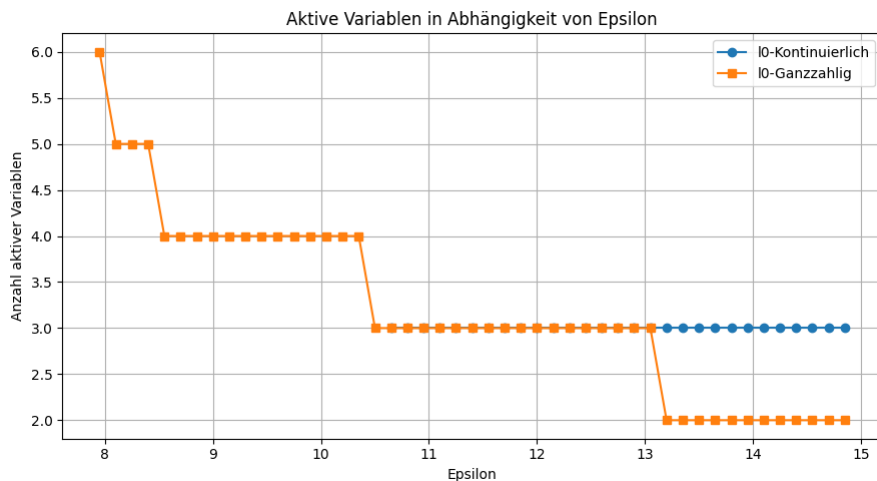


Abbildung 6.1: Vergleich von sparsity gegen Fehlertoleranz im Fall des Einheitskreises mit $\sigma = 0.15$

Fall für $\varepsilon = 10.65$ und im kontinuierlichen Fall mit $\varepsilon = 10.5$ rekonstruiert. Während im ganzzahligen Fall für die nachfolgenden Iterationen immer die richtige Lösung gefunden wurde, ist die Lösung im kontinuierlichen Fall bei der sparsity von zwei zwischen ca. 4 und 9 Prozent relativer Koeffizientenabweichung geschwankt. Dieses Experiment zeigt, dass vor allem das kontinuierliche Modell stark von der Wahl von ε abhängt. Bei niedrigem ε neigt das Modell zum Overfitting und erzielt keine geringe sparsity. Bei sehr großem ε wird dagegen die sparsity gegenüber der Lösungsqualität überbetont. Im Gegensatz dazu ist das Modell mit ganzzahligen Koeffizienten relativ stabil und weniger abhängig von der Wahl von ε .

Als nächstes betrachten wir das Verhalten bei unterbestimmten Problemen am Beispiel der Lemniskate von Gerono. Hierfür haben wir das Verfahren erneut ohne Rauschen für $N \in \{1, \dots, 20\}$ durchgeführt. Um den Einfluss der sparsity zu betrachten haben wir das gleiche Experiment auch für eine Modellvariante ohne sparsity durchgeführt. Die

Modellvariante ohne sparsity löst das Optimierungsproblem

$$\begin{aligned}
& \min \quad \sum_{i=1}^N \varepsilon_i \\
& \text{u. d. N.} \quad -M_{\text{big}} \leq c_j \leq M_{\text{big}}, \quad j = 1, \dots, M, \\
& \quad \quad -\varepsilon_i \leq \sum_{j=1}^M c_j m_j(x^{(i)}) \leq \varepsilon_i, \quad i = 1, \dots, N, \\
& \quad \quad b^\top c = 1, \\
& \quad \quad \varepsilon_i \in \mathbb{R}_+, \quad i = 1, \dots, N, \\
& \quad \quad c_j \in \mathbb{R}, \quad j = 1, \dots, M.
\end{aligned} \tag{6.1}$$

Die letzte Nebenbedingung ändert sich im Fall ganzzahliger Koeffizienten zu $c_j \in \mathbb{Z}$. Den Normierungsvektor b haben wir erneut anhand unserer Pivotwahl bestimmt. Da sich die Laufzeit bei stark unterbestimmten Problemen und falschen Pivots als problematisch herausgestellt hat, haben wir die SCIP-Laufzeit für jedes einzelne MILP auf 30 Sekunden begrenzt und $M_{\text{big}} = 5$ gewählt. Für jede Anzahl von Datenpunkten haben wir dieses Experiment 10 mal durchgeführt. Nach Satz 3.5 sollte für $D = 4$ nach frühestens 14 Iterationen der Kern eindimensional sein. Die Ergebnisse in Abbildung 6.2 zeigen, dass die ganzzahlige Variante mit sparsity schon bei einem einzelnen Datenpunkt in 9 von 10 Durchläufen die korrekte Lösung erkennt. Ab zwei Datenpunkten findet die Variante in allen Fällen die korrekte Gerono-Gleichung. Am zweitbesten eignet sich die Variante mit sparsity und kontinuierlichen Koeffizienten. Diese Variante findet ab drei Datenpunkten in allen Fällen die korrekte Lösung. Die ganzzahlige Variante ohne sparsity benötigt hingegen fünf Datenpunkte, um in allen Fällen die korrekte Lösung zu identifizieren. Die kontinuierliche Variante ohne sparsity benötigt hingegen die in Satz 3.5 berechnete Anzahl an Datenpunkten. Dies zeigt, dass ganzzahlige Koeffizienten dazu führen, dass unterbestimmte Probleme zur richtigen Lösung führen, wenn die Ursprungsgleichung auch ganzzahlig ist. Einen noch größeren Einfluss hat hingegen in unserem Beispiel die sparsity. Die Ergebnisse können allerdings auch damit zusammenhängen, dass im Optimierungsproblem ohne sparsity die Fehlertoleranz minimiert wird und bei unterbestimmten Problemen ohne Rauschen mehrere Gleichungen eine Fehlertoleranz von Null erzielen können.

Als nächstes betrachten wir den Einfluss unserer, in Abschnitt 4.5 hergeleiteten, Presolving-Methoden. Dazu betrachten wir erneut das Beispiel der Lemniskate von Gerono mit

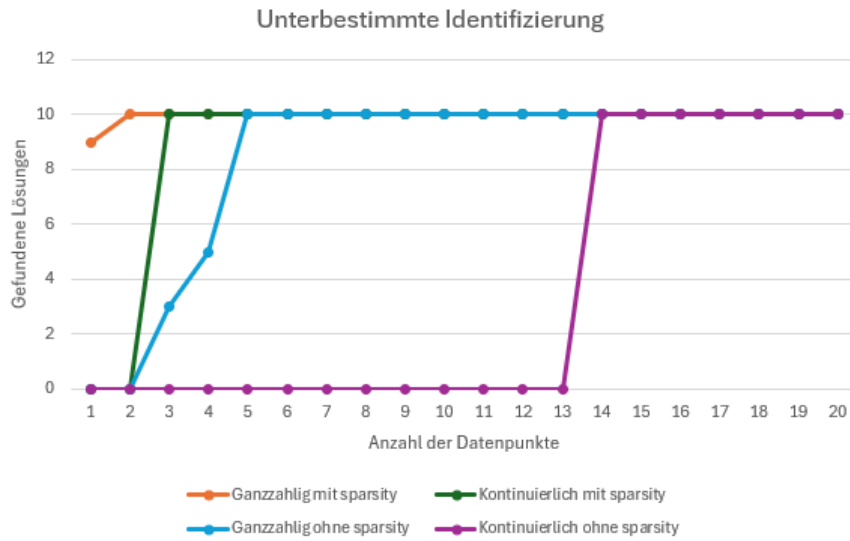


Abbildung 6.2: Unterbestimmte Rekonstruktion der Lemniskate von Geron

der ganzzahligen ℓ_0 -Methode, da diese sich in den bisherigen Experimenten als am zuverlässigsten herauskristallisiert hat. Wir wählen die bisher genutzten Parameter wie bisher und wählen ein Rauschen von 0.02. Für die kernbasierte Monomselektion wählen wir $\sigma_{\text{tol}} = 10^{-2}$, wobei σ_1 der größte Singulärwert ist. Gleichzeitig wählen wir $\tau = 0.1$. Diese Werte haben wir durch vorherige Tests gewählt, da in unseren Tests und in den späteren Ergebnissen mit diesen Werten immer eine große Anzahl an Monomen gestrichen wurden ohne die relevanten Monome zu entfernen. Als Schranke wählen wir die ursprüngliche Schranke $M_{\text{big}} = 1000$. Wir haben erneut so lange getestet, bis 50 mal mit und ohne Presolving die richtige Lösung gefunden wurde. Es wurde jeweils mit den gleichen Stichproben erst ohne Presolving und dann mit Presolving getestet. Die Ergebnisse dieser Tests sind in Tabelle 6.3 zu sehen.

Die Spalten der Tabelle geben von links nach rechts die Lösungszeit des MILP ohne Presolving, die reine MILP-Lösungszeit nach Anwendung des Presolvings, die für das Presolving benötigte Zeit, die daraus resultierende Gesamtzeit, die Reduktion der MILP-Lösungszeit sowie die Reduktion der Branch-and-Bound-Knoten an. Wir haben zunächst die drei Presolving-Methoden einzeln und anschließend in Kombination getestet. Bei den Tests erzielt die Kernreduktion den besten Gesamtkompromiss aus Aufwand und Nutzen. Sie ist dabei die einzige Methode, die auch unter Einbeziehung des Presolving-

Tabelle 6.3: Einfluss der Presolving-Methoden auf Lösungszeit und Branch-and-Bound-Knoten

Methode	t_{ohne}	t_{mit}	t_{pre}	t_{gesamt}	MILP-Red.	Knotenreduktion
Nur ℓ_1	0.256	0.251	0.020	0.272	1.9 %	11.9 %
Nur Kernreduktion	0.254	0.140	0.0004	0.140	45 %	37.1 %
Nur Bounds	0.246	0.125	0.649	0.774	49.4 %	65.4 %
Kombinierte Methoden	0.257	0.105	0.427	0.531	59.3 %	74.4 %

Aufwands eine Reduktion der Gesamtzeit von etwa 45% liefert. Dies kommt vor allem dadurch zustande, dass der Rechenaufwand durch diese Methode gering war. Obwohl die Verfeinerung der Bounds die Anzahl der Branch-and-Bound-Knoten am deutlichsten reduziert und auch die reine MILP-Lösungszeit fast halbiert, verursacht sie einen hohen zusätzlichen Berechnungsaufwand, da im Lösungsprozess für jede Schranke ein LP gelöst werden muss. Zusätzlich war die Reduktion der Lösungszeit und der Knoten so groß, da wir mit einer großen M_{big} -Bedingung gestartet sind. Da die M_{big} mit den Polynomkoeffizienten korrespondieren, wäre für unsere Beispiele von Beginn an eine kleinere M_{big} -Schranke möglich. Der Nutzen der oberen Schranke für die sparsity durch vorherige Berechnung der kontinuierlichen ℓ_1 -Lösung hat insgesamt nur einen geringen positiven Einfluss auf die Lösungszeit des MILP und auf die Anzahl der durchlaufenen Branch-and-Bound-Knoten. Der Vorteil dieser Methode ist, dass nur ein einziges LP gelöst werden muss, wodurch der Rechenaufwand gering ist. Die Kombination aller Methoden führt zwar zur stärksten Reduktion der Knotenanzahl und zur geringsten reinen MILP-Lösungszeit. Diese Vorteile werden durch die zusätzlichen Presolving-Kosten überkompensiert. Trotzdem ist die Presolving Zeit bei der Kombination aller Methoden geringer als bei der Verfeinerung der Bounds alleine. Dies kommt dadurch zustande, dass bei der Kombination der Methoden vor der Berechnung der Schranken schon Monome durch die Kernreduktion entfernt wurden. Dadurch müssen pro entferntem Monom zwei Schranken weniger berechnet werden.

Bemerkung 6.1. Wir haben in unseren Experimenten die bekannte Hyperflächengleichung von der Lemniskate von Gerono, bzw. vom Einheitskreis als Erfolgskriterium genutzt. Im Allgemeinen ist jedoch die korrekte Gleichung nicht a priori bekannt. Bei einer unbekannten Hyperflächengleichung könnte man stattdessen jeweils den ersten Pivot nutzen, für den eine Lösung gefunden wird. Alternativ könnte für jeden Pivot das Problem gelöst werden und daraufhin für jede gefundene Lösung eine gewichtete Summe aus erzielter sparsity und Fehler gebildet werden. Eine weitere Möglichkeit wäre die Stichprobe in Trainings- und Testdaten zu zerlegen und die Lösung zu verwenden, die auf den Testdaten den niedrigsten Fehler aufweist.

Zusammenfassend zeigen die Ergebnisse aus dem Hyperflächenfall, dass unser Optimierungsproblem für kleines oder moderates Rauschen die Hyperflächengleichung zuverlässig bestimmt. Die ganzzahlige ℓ_0 -Variante erzielt die besten und robustesten Ergebnisse. Diese Methode findet ohne Rauschen immer die sparseste Lösung und erweist sich auch bei stärkerem Rauschen als robust, ohne dabei extrem instabil vom Fehlerparameter ε abzuhängen. Allerdings lassen sich mit der ganzzahligen Modellvariante mit unserer Normierungsbedingung $b^\top c = 1$ nur eine relativ kleine Klasse von Problemen lösen. Von unseren Presolving-Methoden hat sich vor allem die Kernreduktion als kostengünstige Variante herausgestellt, um die Lösungszeit und Branch-and-Bound-Knotenanzahl zu reduzieren.

6.2 Allgemeine Varietäten

In diesem Abschnitt testen wir das sequentielle Optimierungsproblem am Beispiel allgemeiner Varietäten. Die Beispiele übernehmen wir aus [4] und vergleichen die Lösungen an einigen Stellen mit den Lösungen aus [4]. Dazu betrachten wir zunächst die spezielle orthogonale Gruppe in drei Dimensionen

$$SO(3) = \{R \in \mathbb{R}^{3 \times 3} : R^\top R = I, \det(R) = 1\}.$$

Die Daten haben wir erzeugt, indem wir ein zufälliges, normiertes Quaternion erzeugt haben und daraus eine Matrix aus $SO(3)$ berechnet haben. Diese Art der Datenerzeugung wurde von Shoemake in [31] beschrieben. Wir verwenden zunächst als Maximalgrad der Monome $D = 2$. Für $D = 2$ ist es möglich 20 linear unabhängige verschwindende Gleichungen zu finden. Wir testen zunächst mit $N = 100$ zufällig verteilten Matrizen aus $SO(3)$ mit $M_{\text{big}} = 20$. In Tabelle 6.4 haben wir für die Rauschlevel $\sigma \in \{0, 0.001, 0.025, 0.1\}$ die gefundenen Lösungen zusammen mit dem Rang der Koeffizientenmatrix und den Lösungszeiten erfasst. Das Rauschen und die Fehlertoleranz haben wir wie in Abschnitt 6.1 erzeugt. Wir haben also Rauschen auf die Residuen

$$0 = f(x_i) + \xi_i, \text{ wobei } \xi_i \sim \mathcal{N}(0, \sigma^2),$$

erzeugt. Da wir im Fall der allgemeinen Varietäten mehrere beobachtete Gleichungen haben, haben wir das Rauschen nicht auf die zu suchende Polynomgleichung addiert, sondern erst im Lösungsprozess als rechte Seite für unser Optimierungsproblem hinzugefügt. Wir suchen also statt $Ax = 0$ die Beziehung

$$Ax = -\xi_i.$$

Da die Ergebnisse des letzten Abschnitts gezeigt haben, dass ganzzahlige Koeffizienten zu stabileren Ergebnissen führen, verwenden wir auch hier ganzzahlige Koeffizienten. Als Presolving-Maßnahme nutzen wir die kernbasierte Monomselektion mit $\sigma_{\text{tol}} = 10^{-3}$ und $\tau = 10^{-2}$, da diese im letzten Abschnitt die einzige Methode mit einem Netto-Nutzen war. Wir haben jeweils das Verfahren abbrechen lassen, wenn die Koeffizientenmatrix den Rang 20 erreicht hat und daraufhin die Gleichungen auf einem unabhängigen rauschfreien $SO(3)$ Datensatz getestet. In allen Fällen wurden nur Gleichungen gefunden, die laut unserer Validierung tatsächlich von $SO(3)$ stammen. Mit einem Blick auf die Ergebnisse

Tabelle 6.4: Vergleich des No-Good-Cut-Verfahrens mit der Pivot-Methode am Beispiel von $SO(3)$ mit $D = 2$

Rauschen	Methode	Gefundene Lösungen	Rang	Zeit in s	Epsilon
0	Pivot	20	20	4.43	0
0	No-Good-Cut	30	20	96.46	0
0.001	Pivot	20	20	20.64	0.089
0.001	No-Good-Cut	30	20	1483.79	0.089
0.025	Pivot	20	20	41.95	2.257
0.025	No-Good-Cut	30	20	1642.31	2.257
0.1	Pivot	20	20	42.04	8.988
0.1	No-Good-Cut	30	20	2370.83	8.988

fällt auf, dass beide Methoden für alle Rauschstärken jeweils alle linear unabhängigen Gleichungen gefunden wurden. Wie in Abschnitt 3.2 erwähnt, zeigt sich auch mit Blick auf die gefundenen Gleichungen im Vergleich zum Rang der Koeffizientenmatrix, dass die Pivot-Methode lineare Unabhängigkeit der gefundenen Gleichungen garantiert. Die No-Good-Cut-Methode hingegen findet für alle Rauschstärken mehr gefundene Gleichungen als maximal linear unabhängig möglich. Es kam dazu, dass zwar korrekte Gleichungen gefunden wurden, diese aber aus anderen Gleichungen linear kombiniert werden konnten.

Mit Blick auf die Lösungszeiten fällt auf, dass die No-Good-Cut-Methode eine deutlich längere Laufzeit aufweist. Dazu haben wir zusätzlich in Abbildung 6.3 die Lösungszeit pro Iteration bei einem Rauschen von 0.025 und $\varepsilon = 2.257$ aufgeschlüsselt. Bei diesem Test fällt der in Abschnitt 3.2 angesprochene Effekt auf. Während bei der Pivot-Methode die Lösungszeit pro Iteration relativ konstant auf einem sehr niedrigen Niveau bleibt, steigt die Lösungszeit bei der No-Good-Cut-Methode mit fast jeder Iteration. Zusätzlich benötigt die No-Good-Cut-Methode 30 statt 20 Iterationen, da sie wie schon angesprochen

keine lineare Unabhängigkeit garantiert. Dies zeigt, wie die langen Lösungszeiten bei der No-Good-Cut-Methode im Vergleich zur Pivot-Methode zustande kommen.

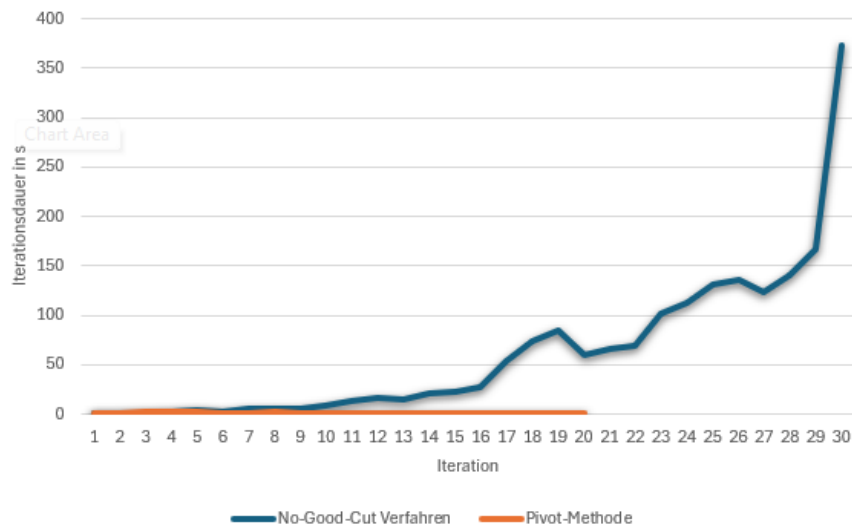


Abbildung 6.3: Vergleich der Iterationszeiten der beiden sequentiellen Verfahren am Beispiel $SO(3)$ bei $\sigma = 0.025$

Als nächstes betrachten wir das gleiche Beispiel der Varietät $SO(3)$ jedoch mit höheren Maximalgraden $D = 2, 3, 4$ und vergleichen die Ergebnisse mit den Verfahren aus [4]. Dazu haben wir analog zu dem zitierten Artikel 887 Beispielmatrizen von $SO(3)$ ohne Rauschen erzeugt und dabei die Anzahl der gefundenen Gleichungen und die Laufzeit der Pivot-Methode in Tabelle 6.5 erfasst. Wir haben in diesem Test auf die No-Good-Cut-Methode verzichtet, da diese für $D = 3$ nach drei Stunden noch nicht alle Gleichungen gefunden hat.

Tabelle 6.5: Gefundene Lösungen mit Pivot-Methode im Vergleich zu Maximalem Polynomgrad $SO(3)$

Maximalgrad D	Gefundene Lösungen	Zeit in s
2	20	34
3	136	410
4	550	7811

Vergleichen wir die gefundenen Ergebnisse mit den Ergebnissen aus [4], so wird deutlich, dass wir mit unserem Verfahren für alle Maximalgrade $D = 2, 3, 4$ alle linear unabhängigen verschwindenden Gleichungen der Varietät $SO(3)$ im betrachteten Polynomraum aufspüren konnten. Mit Blick auf die Lösungszeiten wird allerdings deutlich, dass ein Maximalgrad von $D = 3$ je nach Anwendung mit 410 Sekunden noch gerade so praktikabel ist, während ein Maximalgrad von $D = 4$ zu einer Lösungszeit von mehr als zwei Stunden führt, um alle Gleichungen aufzufinden. Dies liegt vor allem daran, dass bei $D = 4$ eine Gesamtzahl von 715 Monomen erzeugt werden. Dies führt zusätzlich zu der Mehrzahl an Gleichungen auch zu einer großen Anzahl an Binärvariablen. Der Ansatz von Breiding et al. [4] nutzt zur Bestimmung der Lösungen SVD bzw. QR-Zerlegung. Diese Verfahren haben polynomielle Laufzeiten und lassen sich daher für diese Matrizengrößen deutlich schneller durchführen [6]. Im Gesichtspunkt der Lösungszeit kann unser Ansatz deshalb nicht mit dem Ansatz von Breiding et al. mithalten.

In Abschnitt 6.1 hat sich herausgestellt, dass die kernbasierte Monomselektion eine kostengünstige Presolving Variante ist und die Lösungszeit deutlich verbessern kann. Im Fall der Varietät $SO(3)$ führt die Geometrie der Varietät dazu, dass alle Monome in definierenden Gleichungen vorkommen, deshalb kann die kernbasierte Monomselektion keine Monome ausschließen. Ein Beispiel bei dem die kernbasierte Monomselektion deutlich bessere Ergebnisse zeigt ist das zweite Beispiel aus [4], die 2×3 -Matrizen von Rang ≤ 1 . Diese sind für $D = 2$ durch ihre Minoren durch die definierenden Gleichungen

$$x_1x_4 - x_2x_3 = 0, \quad x_1x_6 - x_2x_5 = 0, \quad x_3x_6 - x_4x_5 = 0$$

eindeutig bestimmt. Für jeden Datensatz haben wir zwei zufällige, normalverteilte Vektoren $u \in \mathbb{R}^2$ und $v \in \mathbb{R}^3$ und über das Produkt

$$M = uv^\top$$

zu einer 2×3 -Matrix kombiniert. In Tabelle 6.6 sind die Ergebnisse unseres Verfahrens für die gleichen Rauschstärken wie im Fall von $SO(3)$ abgedruckt. Auffällig ist, dass die kernbasierte Monomselektion mit $\tau = 0.01$ und $\sigma_{\text{tol}} = 0.001$ viele nicht relevante Monome schon vor der Optimierung erkennt und ausschließt. Somit kann die Gesamtzahl der Monome deutlich reduziert werden. Allerdings haben wir mit unserem Rauschmodell nicht die Matrixeinträge der Rang-1 2×3 -Matrizen verrauscht, sondern ausschließlich die Residuen. Dies führt dazu, dass die kernbasierte Monomselektion auf unverrauschten Daten arbeitet, da wir das Rauschen erst im Optimierungsproblem hinzugefügt haben. Deshalb haben wir zusätzlich ein Rauschmodell getestet, bei dem die einzelnen Matrixeinträge verrauscht werden. Wir ersetzen also $x^{(i)}$ durch

$$x^{(i)} + \xi_i, \text{ wobei } \xi_i \sim \mathcal{N}(0, \sigma^2).$$

Tabelle 6.6: Vergleich des No-Good-Cut-Verfahrens mit der Pivot-Methode am Beispiel von Rang 1 2×3 -Matrizen mit $D=2$

Rauschen	Methode	Gefundene Lösungen	Entfernte Monome	Zeit in s	Epsilon
0	Pivot	3	22	0.09	0
0	No-Good-Cut	3	22	0.19	0
0.001	Pivot	3	22	0.11	0.089
0.001	No-Good-Cut	3	22	0.34	0.089
0.025	Pivot	3	22	0.37	2.247
0.025	No-Good-Cut	3	22	0.83	2.247
0.1	Pivot	3	22	0.88	8.988
0.1	No-Good-Cut	3	22	2.72	8.988

Dieses Rauschmodell entspricht einem Messrauschen beim Messen der Matrixeinträge. Der Ansatz ist nicht mit der a priori Schätzung von ε mit der Monte-Carlo-Methode vereinbar. Deshalb haben wir ε in diesem Fall durch Testen verschiedener Werte ermittelt. Ebenso haben wir τ und σ_{tol} durch Testen verschiedener Werte ermittelt. Die Ergebnisse sind in Tabelle 6.7 zu sehen.

Tabelle 6.7: Vergleich des No-Good-Cut-Verfahrens mit der Pivot-Methode am Beispiel von Rang 1 2×3 -Matrizen mit $D=2$ und verrauschten Matrixeinträgen

Rauschen	Methode	Gefundene Lösungen	Entfernte Monome	τ/σ_{tol}	Epsilon
0	Pivot	3	22	0.01/0.001	0
0	No-Good-Cut	3	22	0.01/0.001	0
0.001	Pivot	3	22	0.01/0.001	0.3
0.001	No-Good-Cut	3	22	0.01/0.001	0.3
0.025	Pivot	3	22	0.02/0.1	7
0.025	No-Good-Cut	3	22	0.02/0.1	7
0.1	Pivot	3	13	0.035/0.2	25
0.1	No-Good-Cut	3	13	0.035/0.2	25

Aus den Ergebnissen geht hervor, dass eine deutlich größere Fehlertoleranz ε nötig war, als bei verrauschten Residuen. Zusätzlich ist die kernbasierte Monomselektion im heuristischen Fall, dass die Matrixeinträge verrauscht sind, deutlich mehr von den richtigen Werten von τ und σ_{tol} abhängig. Während für kleines und mittleres Rauschen noch alle 22 nicht

relevanten Monome identifiziert werden konnten, war die Trennlinie zwischen relevanten und nicht relevanten Monomen für ein Rauschen von $\sigma = 0.1$ deutlich unschärfer. Dies führt dazu, dass in diesem Fall nur noch 13 Monome entfernt werden konnten.

Abschließend können wir festhalten, dass sich sowohl die No-Good-Cut-Methode als auch die Pivot-Methode in unseren Experimenten grundsätzlich als geeignet herausgestellt haben. Insgesamt ist die Pivot-Methode aufgrund der kürzeren Iterationszeiten und der garantierten linearen Unabhängigkeit das deutlich praktikablere sequentielle Verfahren. Außerdem haben wir die Grenzen der kernbasierten Monomselektion erkennen können. Diese tritt bei Daten auf, bei denen alle Monome an verschwindenden Polynomgleichungen beteiligt sind.

6.3 Symbolische Regression

Abschließend testen wir unseren Algorithmus zur Lösung des Problems der symbolischen Regression. Dazu nutzen wir die 100 Gleichungen aus [34] zum Testen unseres Algorithmus. Als Bausteine unserer Funktionsbibliothek nutzen wir die $\sin$, $\cos$ und die Inverse. Aus diesen Bausteinen erzeugen wir die Features unserer Bibliothek wie in Abschnitt 5.1 erläutert. Da unser Algorithmus mit einer Funktionsbibliothek arbeitet, ist es für den Erfolg des Algorithmus relevant, dass die Gleichungen als Linearkombination von Funktionsbausteinen unserer Bibliothek darstellbar sind. Mit unseren Funktionsbausteinen sind 53 der 100 Gleichungen theoretisch darstellbar, deshalb haben wir uns auf diese 53 Gleichungen beschränkt. Es wäre möglich gewesen noch einige weitere der Feynman-Gleichungen zu lernen, indem wir die Funktionsbibliothek deutlich vergrößern. Der Anspruch dieser Arbeit ist jedoch nicht alle 100 Gleichungen zu lernen. Stattdessen war das Ziel eine performante Version zu erarbeiten, die zuverlässig Gleichungen auch unter Rauschen identifizieren kann, wenn sich die Grundbausteine in der Funktionsbibliothek befinden.

Dies bringt uns zum nächsten Punkt. Ähnlich wie in [34] haben wir unseren Algorithmus auch an verrauschten Daten getestet. Dies ist in diesem Fall besonders interessant, da es sich bei den Feynman-Gleichungen um Gleichungen aus der Physik handelt. Die Daten können von Messgeräten stammen, bei denen eine gewisse Messungenauigkeit üblich ist. Wir haben zunächst die Funktion ohne Rauschen getestet. Daraufhin haben wir mit Rauschen in Form von einer Standardabweichung von $\sigma = 10^{-3}$ auf den Daten gestartet und jeweils das Rauschen um den Faktor 10 vergrößert, bis die Gleichung nicht mehr

identifiziert werden konnte. Wir erzeugen also Daten

$$y_i = f(x_i) + \xi_i, \text{ wobei } \xi_i \sim \mathcal{N}(0, \sigma^2).$$

In der Tabelle 6.8 haben wir das größtmögliche Rauschen, bei dem die richtige Lösung gefunden wurde. Die Fehlertoleranz ε haben wir erneut mithilfe von Monte-Carlo-Simulation bestimmt. Die Vorgehensweise dabei haben wir in Abschnitt 6.1 erläutert. Ein Spezialfall waren die vier mit * beziehungsweise ** markierten Gleichungen in Tabelle 6.8, da sie erst nach einer Zielraumtransformation von unserer Funktionsbibliothek darstellbar waren. Bei diesen Beispielen haben wir nicht die Beziehung

$$y = f(x)$$

gelernt. Stattdessen haben wir die Beziehung

$$y_i^2 = (f(x_i))^2, \text{ bzw. } \ln(y_i) = \ln(f(x_i))$$

gelernt. Da wir die Transformation nach Hinzufügen des Rauschens durchführen, transformieren wir unter Rauschen

$$y^2 = (f(x) + \xi_i)^2, \text{ bzw. } \ln(y) = \ln(f(x) + \xi_i).$$

Das Rauschen hängt somit vom Funktionswert ab und unser Monte-Carlo-Ansatz funktioniert an dieser Stelle nicht. Deshalb haben wir die Fehlertoleranz in diesen Fällen durch Testen verschiedener Fehlertoleranzen bestimmt.

Zur Berechnung des argmin in (5.3) nutzen wir die Idee aus [35]. Dazu schreiben wir den Betrag wie folgt um:

$$\sum_{i=1}^N |\phi_{ij}c - y_i| = \sum_{i \in I_j} |\phi_{ij}| \left| c - \frac{y_i}{\phi_{ij}} \right| + \sum_{i \notin I_j} |y_i|. \quad (6.2)$$

Hierbei definieren wir

$$I_j := \{i \in \{1, \dots, N\} \mid \phi_{ij} \neq 0\}.$$

Der zweite Summand in (6.2) ist konstant bezüglich c und spielt deshalb keine Rolle für das argmin. Den Spezialfall $I_j = \emptyset$ müssen wir nicht betrachten, da wir Features aus der Bibliothek entfernen, die auf allen Punkten null sind. Damit entspricht das Problem

$$\min_{c \in [-M_{\text{big}}, M_{\text{big}}]} \sum_{i=1}^N |\phi_{ij}c - y_i|$$

für ein festes Feature der Minimierung einer Summe gewichteter absoluter Abweichungen. Ein gewichteter Median der Werte $\frac{y_i}{\phi_{ij}}$ mit den Gewichten $|\phi_{ij}|$ minimiert die Summe der gewichteten absoluten Abweichungen [35]. Liegt ein gewichteter Median außerhalb des zulässigen Intervalls $[-M_{\text{big}}, M_{\text{big}}]$ projizieren wir ihn auf die nächstgelegene Intervallgrenze.

Wir haben zunächst den Algorithmus mit der vollen Funktionsbibliothek mit den drei Funktionsbausteinen und einem maximalen Grad der Monome von $D = 2$ und einem $M_{\text{big}} = 20$ mit und ohne Presolving getestet. Hierbei haben wir unabhängig und gleichverteilt für jede Variable Daten im Bereich von $[-2, 2]$ erzeugt. Eine einzige Ausnahme haben wir bei der Gleichung

$$c = \sqrt{\frac{\gamma pr}{\rho}}$$

gemacht. Bei dieser Gleichung haben wir die Eingabedaten im Bereich $[0, 4]$ erzeugt, da wir sonst die Gefahr hatten, dass die Wurzel nicht definiert ist. Daraufhin haben wir die Eingabedaten auf den jeweiligen Gleichungen ausgewertet, um die Evaluationsmatrix für unser MILP zu erzeugen. Falls bei der Lösung des MILP nach einer Stunde keine Lösung gefunden werden konnte, haben wir den Versuch mit der vollen Bibliothek als gescheitert erklärt. Daraufhin haben wir die Funktionsbibliothek durch manuelle Eingriffe auf ein Minimum reduziert, sodass wir möglichst viele der Grundbausteine entfernt haben. Nun haben wir mit dieser reduzierten Bibliothek einen erneuten Versuch gestartet. Wurde erneut nach einer Stunde keine Lösung gefunden, haben wir die Anzahl der Daten um den Faktor 10 erhöht. Daraufhin haben wir den Test mit der gleichen Funktionsbibliothek und der gleichen Datenanzahl wiederholt. Bei diesem Versuch haben wir den Test nach einer überschrittenen Maximalzeit von fünf Stunden als gescheitert erklärt.

Da die Funktionen aus der Feynman-Bibliothek sehr häufig keine ganzzahligen Parameter haben, wählten wir kontinuierliche Parameter. Wie bei den sonstigen numerischen Experimenten, haben wir eine Lösung als korrekt gewertet, wenn der korrekte Support gefunden wurde und die relative Abweichung der Parameter weniger als 5% betrug.

Tabelle 6.8: Ergebnisse bei Tests an den Feynman-Gleichungen aus [34]; *: Konnte nur nach Quadrieren der Testdaten gefunden werden; **: Konnte nur nach logarithmieren der Testdaten gefunden werden

Gleichung	Lösungszeit mit/ohne Presolving	Lösungszeit mit Rauschen mit/ohne PS	Rausch-tol.	volle Bib. mit /ohne PS
1. $f = \frac{\exp(-\frac{\theta^2}{2})}{\sqrt{2\pi}}$ **	1/1	1/2	0.001	ja
2. $F = q_2 E_f$	1/4	1/6	0.1	ja
3. $E = h\omega$	1/4	1/6	0.1	ja
4. $F = \mu N_n$	1/4	1/6	0.1	ja
5. $L = nh$	1/4	1/6	0.1	ja
6. $U = \frac{k_{\text{spring}} x^2}{2}$	1/2	1/17	0.1	ja
7. $k = \frac{\omega}{c}$	1/2	1/4	0.1	ja
8. $V_e = \frac{q}{C}$	1/2	1/4	0.1	ja
9. $E = \frac{3}{2} p_F V$	1/5	1/33	0.1	ja
10. $F_E = \frac{P}{4\pi r^2}$	1/1	1/2	0.01	ja
11. $E_{\text{den}} = \frac{\epsilon E_f^2}{2}$	1/2	1/18	0.1	ja
12. $E_{\text{den}} = \epsilon E_f^2$	1/2	1/3	0.1	ja
13. $U = mgz$	1/11	1/931	0.1	ja
14. $D = \mu_e k_b T$	1/11	1/931	0.1	ja
15. $\tau = r F \sin(\theta)$	1/6	1/55	0.1	ja
16. $V_e = \frac{q}{4\pi\epsilon r}$	1/6	1/322	0.01	ja
17. $I = \frac{qv}{2\pi r}$	1/6	1/11	0.1	ja
18. $\mu_M = \frac{qvr}{2}$	1/8	1/200	0.1	ja
19. $\mu_M = \frac{qh}{4\pi m}$	1/11	1/759	0.01	ja
20. $\omega = \frac{2\mu_M B}{h}$	1/46	1/67	0.1	ja
21. $v = \frac{2E_d^2 k}{h}$	1/109	1/142	0.001	ja
22. $E = 2U(1 - \cos(kd))$	184/184	3547/10827	0.1	ja
23. $f = \beta(1 + \alpha \cos(\theta))$	342/461	2929/14922	0.1	ja
24. $E = \frac{3}{5} \frac{q^2}{4\pi\epsilon d}$	1/20	1/216	0.01	ja
25. $E = -\mu_M B \cos(\theta)$	1/8	1/862	0.1	ja
26. $E = -p_d E_f \cos(\theta)$	1/8	1/862	0.1	ja
27. $F_E = \epsilon c E_f^2$	1/23	1/70	0.1	ja
28. $E = \mu_M(1 + \chi)B$	126/219	6249/9640	0.1	ja
29. $F = \frac{YA_x}{d}$	1/20	1/218	0.1	ja

Gleichung	Lösungszeit mit/ohne Presolving	Lösungszeit mit Rauschen mit/ohne PS	Rausch-tol.	volle Bib.
30. $m = \frac{h^2}{2Ed^2}$	1/22	1/27	0.1	nein
31. $k = \frac{2\pi\alpha}{nd}$	1/19	1/152	0.1	ja
32. $E_f = \frac{q_1}{4\pi\epsilon r^2}$	1/73	1/356	0.1	ja
33. $K = \frac{1}{2}m(v^2 + u^2 + w^2)$	8/3	10/45	0.1	nein
34. $L = mrv \sin(\theta)$	1/38	1/699	0.1	nein
35. $\omega = \frac{qvB}{p}$	1/13	1/1077	0.1	nein
36. $P_F = \frac{nk_b T}{V}$	1/10	1/1077	0.1	nein
37. $v = \frac{\mu_{\text{drift}} q V_e}{d}$	1/10	1/1077	0.1	nein
38. $\omega = \frac{g - qB}{2m}$	1/7	1/48	0.1	nein
39. $B = \frac{1}{4\pi\epsilon c^2} \frac{2I}{r}$	1/1047	1/107	0.01	nein
40. $j = \frac{-\rho_{c0} q A \text{vec}}{m}$	1/5	1/12	0.1	nein
41. $F = \frac{q_1 q_2}{4\pi\epsilon r^2}$	1/139	1/2663	0.1	ja
42. $E = \frac{1}{4}m(\omega^2 + \omega_0^2)x^2$	6/20	14/17	0.01	nein
43. $P = \frac{q^2 a^2}{6\pi\epsilon c^3}$	1/66	1/101	0.01	nein
44. $r = \frac{4\pi\epsilon\hbar}{mq^2}$	1/23	1/39	0.1	nein
45. $x = \mu_m \sqrt{B_x^2 + B_y^2 + B_z^2}$	2/19	41/12	0.01	nein
46. $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$	54/60	623/688	0.01	nein
47. $c = \sqrt{\frac{\gamma p r}{\rho}}$	1/15	1/46	0.01	nein
48. $F = q(E_f + Bv \sin(\theta))$	–	–	–	–
49. $P = \frac{\kappa(T_2 - T_1)A}{d}$	635/–	–	0.1	nein
50. $E = \frac{g - \mu_M B J_z}{h}$	1/1480	1/–	0.01	nein
51. $U = Gm_1 m_2 (\frac{1}{r_2} - \frac{1}{r_1})$	3465/–	–/–	–	nein
52. $P_* = \frac{n_p p_d^2 E_f}{3k_b T}$	1/759	1/10265	0.01	nein
53. $A = x_1 y_1 + x_2 y_2 + x_3 y_3$	388/440	2594/–	0.1	nein

Insgesamt zeigt ein Blick auf die Tabelle, dass fast alle Gleichungen, die theoretisch mit unserer Funktionsbibliothek darstellbar waren, innerhalb der Zeitrestriktionen von unserem Algorithmus gelöst werden konnten. Eine einzige Ausnahme stellt dabei die Gleichung $F = q(E_f + Bv \sin(\theta))$ dar, die weder mit, noch ohne Presolving gelöst werden konnte. Bei allen Gleichungen außer Gleichung 49 und 51 haben 10 Datenpunkte zur Rekonstruktion der Gleichungen mit und ohne Rauschen ausgereicht. Bei den zwei genannten Gleichungen wurden 100 Datenpunkte benötigt. Die Ergebnisse deuten darauf hin, dass die Laufzeit ohne Presolving insbesondere mit der Anzahl der

Variablen und der daraus resultierenden Größe der Kandidatenbibliothek zunimmt. Gleichungen mit ein oder zwei Variablen konnten jeweils immer mit der vollen Kandidatenbibliothek in wenigen Sekunden bestimmt werden. Gleichungen mit drei Variablen konnten bis auf eine Ausnahme auch mit der vollen Kandidatenbibliothek gelöst werden. Bei der Ausnahme mit Gleichung 30 musste die Kandidatenbibliothek angepasst werden, da die Gleichung nur bei einem Maximalgrad der Monome von $D = 3$ darstellbar war. Ab einer Variablenanzahl von vier konnten nur noch wenige Gleichungen mit der ursprünglichen Funktionsbibliothek ohne Presolving gelöst werden. Zum einen liegt das daran, dass mit einer größeren Variablenanzahl auch die Funktionsbibliothek stark ansteigt. Dies ist allerdings nicht die einzige Erklärung. Die reduzierte Funktionsbibliothek bei der Gleichung

$$A = x_1y_1 + x_2y_2 + x_3y_3$$

hat nur 414 Kandidaten. Dies ist vergleichbar mit den 465 Kandidaten der vollen Funktionsbibliothek bei zwei Variablen. Trotzdem ist die Lösungszeit deutlich größer als die Lösungszeit aller Beispiele mit zwei Variablen. Ein weiterer Treiber für die Laufzeit war die Anzahl der aktiven Features in der letztendlichen Lösung. Die Gleichung

$$E = \mu_M(1 + \chi)B$$

hat sich im Vergleich zu anderen Funktionen mit drei Variablen als besonders schwierig herausgestellt. Bei dieser Gleichung waren in der letztendlichen Lösung zwei Features aktiv im Vergleich zu den vielen anderen Beispielen mit einem aktiven Feature. Im Hinblick auf die Laufzeit fällt zusätzlich auf, dass etwa 80% der Gleichungen ohne Rauschen und Presolving in unter 60 Sekunden gelöst werden können. Im Gegensatz dazu machen die langsamsten drei Instanzen etwa 61% der gesamten aufsummierten Laufzeit ohne Rauschen und Presolving aus.

Mit Blick auf die Presolving-Maßnahmen ist auffällig, dass alle Gleichungen, die ohne MILP gelöst werden konnten, in einer Sekunde mit und ohne Rauschen rekonstruiert werden konnten. Dies betrifft Gleichungen, die mit einem einzigen Feature unserer Kandidatenbibliothek gelöst werden können. Dies führt insbesondere bei Rauschen in vielen Fällen zu einer deutlichen Reduktion der Lösungszeit. Die Größe der Kandidatenbibliothek hat auf diese Instanzen mit Presolving keinen großen Einfluss gehabt. Deshalb könnten mit Presolving auch mehr als die benannten Instanzen mit der vollen Kandidatenbibliothek gelöst werden. Um allerdings eine Vergleichbarkeit der Lösungszeiten zu gewährleisten, haben wir die Kandidatenbibliothek jeweils für beide Verfahren reduziert. Im besten Fall hat die Variante mit Presolving bei Gleichung 52 mit Rauschen nur 0.0097% der Lösungszeit ohne Presolving benötigt. Doch auch bei Gleichungen, die aus mehreren Kandidaten linear kombiniert werden, konnte die Lösungszeit in vielen Fällen deutlich reduziert werden. Gleichung 53 konnte mit Rauschen beispielsweise ohne Presolving innerhalb der Zeitrestriktion nicht rekonstruiert werden, während die Variante mit Presolving unter eine Stunde zur Rekonstruktion benötigt hat. Die Gesamtlösungszeit ohne Rauschen konnte für die 50 Instanzen, die von beiden Varianten rekonstruiert werden konnten, durch das Presolving um 79% reduziert werden.

Unter Rauschen waren vor allem Beispiele mit sin oder cos besonders schwer zu identifizieren. Dies lässt sich möglicherweise durch den periodischen Verlauf trigonometrischer Funktionen erklären.

Der Solver muss dadurch zwischen vielen strukturell sehr ähnlichen Kandidaten unterscheiden. Dies führt zu schwachen Schranken innerhalb des Branch-and-Bound Verfahrens. Weiterhin haben sich Brüche mit einem großen Nenner unter Rauschen als schwer identifizierbar herausgestellt. Dies ist dadurch zu erklären, dass die Funktionswerte hier nahe Null sind, wodurch das Rauschen im Vergleich zu den Funktionswerten einen großen Einfluss hat. Zusätzlich haben Brüche auch das Problem bei Daten, die einen Nenner nahe Null erzeugen. Dies führt zu einzelnen Ausreißern und einer schlechten numerischen Konditionierung. Ein Spezialfall waren die Gleichungen, bei denen eine Zielraumtransformation nötig war. In diesen Fällen wurde das Rauschen durch die Transformation teilweise verstärkt, wodurch eine deutlich niedrigere Rauschtoleranz das Resultat war. Zusätzlich hat hier unsere Idee zur Ermittlung der Fehlertoleranz ε nicht gegriffen.

Wie schon besprochen, konnten wir im Gegensatz zu [34] nicht alle Gleichungen bestimmen, da wir von unserer Funktionsbibliothek abhängig sind. Die Autoren dieser Arbeit nutzen gezielt die physikalischen Einheiten, multiplikative und additive Separabilität, um das Grundproblem in leichter lösbare Teilprobleme zu zerteilen. Dadurch konnten in dieser Arbeit alle Gleichungen bestimmt werden. Die Ergebnisse zeigen jedoch, dass unser Verfahren in dem hier untersuchten Versuchsaufbau bei vergleichsweise starkem Rauschen mehrere Gleichungen korrekt identifizieren konnte. Ein direkter Vergleich ist jedoch nicht in allen Fällen möglich, da wir zur Reduktion unserer Kandidatenbibliothek Vorwissen genutzt haben. Für einen direkten Vergleich müsste ein Algorithmus zur automatischen Reduktion der Kandidatenbibliothek erstellt werden.

7 Fazit

Ziel dieser Arbeit war es, sparse und damit gut interpretierbare algebraische Gleichungen anhand von möglicherweise verrauschten Beispieldaten zu lernen. Dazu haben wir zunächst algebraische Varietäten betrachtet. Hierbei haben wir Definitionen und Resultate der algebraischen Geometrie wiederholt. Diese Aussagen bilden die theoretische Grundlage, um eine endliche Beschreibung einer Varietät zu erzielen. Aufbauend auf diesen Resultaten haben wir ein Optimierungsproblem hergeleitet, mit dem Polynome bestimmt werden können, die auf den gegebenen Stichproben verschwinden. Um aus den möglichen Polynomgleichungen interpretierbare Gleichungen auszuwählen, haben wir eine sparsity-Bedingung minimiert. Zur Modellierung dieser sparsity-Bedingung haben wir eine Big-M-Bedingung zur Modellierung als MILP genutzt. Zunächst haben wir den Spezialfall von Hyperflächen betrachtet. Hier genügt eine einzelne Polynomgleichung zur Beschreibung der Nullstellenmenge der Varietät. Daraufhin haben wir durch die Formulierung der Pivot-Methode und der No-Good-Cut-Methode zwei sequentielle Verfahren hergeleitet. Dadurch haben wir das Optimierungsproblem auf allgemeine Varietäten erweitert, die durch mehrere Gleichungen beschrieben werden. Die Pivot-Methode erzwingt lineare Unabhängigkeit der gefundenen Gleichungen, während die No-Good-Cut-Methode lediglich bereits gefundene Supports ausschließt. In den numerischen Experimenten hat sich die Pivot-Methode als das geeignetere Verfahren herausgestellt. Die Lösungszeiten der einzelnen Iterationen blieben bei der Pivot-Methode vergleichsweise konstant, während sie bei der No-Good-Cut-Methode mit der Anzahl der hinzugefügten Ausschlussbedingungen deutlich anstiegen. Zusätzlich erzeugte die No-Good-Cut-Methode linear abhängige Gleichungen mit unterschiedlichen Supportmustern.

Im Anschluss an die Herleitung der Optimierungsprobleme haben wir deren Komplexität mithilfe linearer Algebra untersucht. Mithilfe des Kerns der Evaluationsmatrix konnten wir Kriterien für die Existenz und Eindeutigkeit einer Lösung herleiten. Insbesondere gibt die Dimension des Kerns an, wie viele linear unabhängige Polynomgleichungen im betrachteten Polynomraum auf den Daten verschwinden können. Für den Fall von Hyperflächen haben wir außerdem eine notwendige Mindestanzahl an Stichproben für die eindeutige Bestimmung einer Gleichung bis auf Skalierung hergeleitet. Weiterhin haben wir durch eine Reduktion von einem bekannten Problem gezeigt, dass das zugrundeliegende Entscheidungsproblem zu unserer Problemformulierung NP-schwer ist. Die NP-Schwere entsteht durch die sparsity-Bedingung. Daraufhin haben wir Faktoren untersucht, die das Verhalten eines Branch-and-Bound-Solvers in der Praxis beeinflussen. Auf Grundlage dieser Analyse haben wir verschiedene Maßnahmen entwickelt, um die Komplexität des Optimierungsproblems vor dem Lösungsprozess zu reduzieren. Dazu gehören ein ℓ_1 -Warmstart,

eine Verfeinerung der Big-M-Bedingung durch das Lösen von linearen Programmen und eine kernbasierte Monomselektion. Die kernbasierte Monomselektion hat sich in den numerischen Experimenten als wirksam herausgestellt, vor allem wenn nur ein kleiner Teil der ursprünglichen Monombasis für die Beschreibung der Varietät benötigt wurde. Ihre Wirksamkeit hängt allerdings von der Struktur des betrachteten Problems ab. Bei den betrachteten Hyperflächen und den 2×3 -Matrizen von Rang ≤ 1 konnte die Anzahl der Monome deutlich reduziert werden. Bei $SO(3)$ konnten dagegen keine Monome entfernt werden, da jedes der betrachteten Monome in einer möglichen verschwindenden Gleichung vorkam. Die Presolving-Maßnahme ist daher nicht für jede Varietät geeignet.

Insgesamt konnten bei rauschfreien Daten und bei geringem Rauschen sowohl Hyperflächengleichung als auch mehrere Gleichungen allgemeiner Varietäten in den betrachteten Beispielen zuverlässig rekonstruiert werden. Bezüglich der Anzahl der linear unabhängigen Gleichungen kann unser Verfahren damit mit dem Verfahren aus [4] mithalten, während unser Verfahren zusätzlich sparse Gleichungen bevorzugt. Bei stärkerem Rauschen hat sich die Modellvariante mit ganzzahligen Koeffizienten als besonders robust herausgestellt. In den Experimenten reduzierten die ganzzahligen Koeffizienten die Gefahr des Overfittings und führten dazu, dass auch unterbestimmte Evaluationsmatrizen zu einer Rekonstruktion der Gleichung genühten. Diese Variante kann allerdings nur verwendet werden, wenn die Annahme ganzzahliger Koeffizienten für die gesuchte Gleichung sinnvoll ist und die Skalierungsbedingung $b^\top c = 1$ erfüllbar ist. Die Skalierungsbedingung ist im Fall ganzzahliger Koeffizienten eine deutlich größere Einschränkung als im kontinuierlichen Fall. Die Experimente haben außerdem gezeigt, dass die Wahl des Fehlerbudgets ε einen wesentlichen Einfluss auf die gefundene Gleichung hat. Wird ε zu klein gewählt, kann die korrekte Gleichung bei verrauschten Daten unzulässig werden. Bei einem zu großen Fehlerbudget bevorzugt der Solver durch die sparsity-Minimierung einfache Gleichungen, die zwar das Fehlerbudget einhalten, jedoch auf der zugrunde liegenden Varietät nicht verschwinden.

Zusätzlich zur Rekonstruktion algebraischer Varietäten haben wir untersucht, wie sich der entwickelte Ansatz auf das Grundproblem der symbolischen Regression übertragen lässt. Durch kleine Anpassung des Optimierungsproblems für Hyperflächen konnten wir ein Verfahren zur Rekonstruktion physikalischer Gleichungen aus Stichproben formulieren. Hierzu haben wir ein Verfahren zum Erzeugen einer Kandidatenbibliothek hergeleitet. Aufgrund dieser Bibliothek konnten die Koeffizienten der gesuchten Gleichungen erneut durch ein sparsity-basiertes MILP bestimmt werden. Zusätzlich haben wir Presolving-Maßnahmen für die symbolische Regression hergeleitet. Diese beinhalten die Erkennung von Gleichungen, die von einem einzigen Kandidaten der Bibliothek erzeugt werden und eine Reduktion der Kandidatenbibliothek. Unter Anwendung des Presolvings konnten 52 der 53, mit unserer Kandidatenbibliothek darstellbaren, Feynman-Gleichungen rekonstruiert werden. Insbesondere Gleichungen mit wenigen Variablen und kleinen Supports konnten zuverlässig, auch ohne Presolving, mit kurzen Laufzeiten bestimmt werden. Fast alle Gleichungen ließen sich auch bei verrauschten Daten korrekt rekonstruieren. Insbesondere das Presolving hat zu einer deutlichen Reduktion der Rechenkomplexität geführt. Gleichungen, die von einem einzigen Kandidaten erzeugt wurden, konnten mit Presolving in einer Sekunde rekonstruiert werden. Auch bei den sonstigen Gleichungen hat das Presolving vermehrt eine deutliche Reduktion

der Laufzeit erzielt. Eine wesentliche Grenze des Ansatzes besteht in der Abhängigkeit von der vorgegebenen Kandidatenbibliothek. Eine Gleichung kann nur rekonstruiert werden, wenn die benötigten Funktionsbausteine in der Bibliothek enthalten sind. Gleichzeitig führt eine zu große Kandidatenbibliothek zu einer größeren Anzahl kontinuierlicher und binärer Variablen und erschwert dadurch die Lösung des MILPs. Es besteht somit ein Konflikt zwischen einer möglichst umfangreichen Bibliothek und einem möglichst kleinen MILP. Eine weitere Grenze besteht vor allem ohne Presolving darin, dass nur Gleichungen bis zu einer Variablenanzahl von drei konstant ohne manuelle Reduktion der Kandidatenbibliothek rekonstruiert werden können. Durch Presolving kann diese Schwäche jedoch abgeschwächt werden, da zumindest Gleichungen, die aus einem Kandidaten linear kombiniert werden können, konstant schnell rekonstruiert werden.

Ein Anknüpfungspunkt für zukünftige Arbeiten besteht daher in einer verbesserten oder adaptiven Kandidatenbibliothek. Statt alle Kandidaten von Beginn an in das Optimierungsproblem aufzunehmen, könnten neue Funktionsbausteine schrittweise erzeugt oder anhand der Daten priorisiert werden. Zusätzlich könnte die in [34] verwendete Ausnutzung der physikalischen Einheiten sowie die additive und multiplikative Separabilität als Presolving-Maßnahme für unser Problem genutzt werden. Nach den Zerlegungsschritten aus [34] könnte unser MILP eingesetzt werden, um die zerlegten Funktionsbausteine zu lernen. Dadurch könnten die Vorteile beider Verfahren genutzt werden. Somit könnte das Verfahren auch auf Probleme mit mehr Variablen, mehr aktiven Funktionsbausteinen und stärkerem Rauschen angewandt werden. Zusätzlich ähnelt unser Ansatz zur symbolischen Regression nach Erstellung der Kandidatenbibliothek und Auswertung an den Datenpunkten bei wenigen vorhandenen Stichproben dem Grundproblem des Compressed Sensing [7]. Auch beim Compressed-Sensing wird ein sparser Koeffizientenvektor aus einem unterbestimmten linearen Gleichungssystem rekonstruiert. Deshalb könnte hier ein Anknüpfungspunkt für weitere Arbeiten sein, die Theorie des Compressed Sensing zu untersuchen und die Anwendbarkeit auf unser Problem zu analysieren.

Literatur

- [1] E. Beale und John Tomlin. „Special Facilities in a General Mathematical Programming System for Non-Convex Problems using Ordered Sets of Variables“. In: *Operational Research* 69 (Jan. 1969), S. 447–454.
- [2] Dimitris Bertsimas, Angela King und Rahul Mazumder. *Best Subset Selection via a Modern Optimization Lens*. 2015. arXiv: 1507.03133 [stat.ME]. URL: <https://arxiv.org/abs/1507.03133>.
- [3] Suresh Bolusani u. a. *The SCIP Optimization Suite 9.0*. 2024. arXiv: 2402.17702 [math.OC]. URL: <https://arxiv.org/abs/2402.17702>.
- [4] Paul Breiding u. a. „Learning Algebraic Varieties from Samples“. In: *Revista Matemática Complutense* 31.3 (2018), S. 545–593. DOI: 10.1007/s13163-018-0273-6.
- [5] James W Demmel. *Applied Numerical Linear Algebra*. SIAM, 1997. DOI: 10.1137/1.9781611971446.
- [6] Jack Dongarra u. a. „The Singular Value Decomposition: Anatomy of Optimizing an Algorithm for Extreme Scale“. In: *SIAM review* 60.4 (2018), S. 808–865. DOI: 10.1137/17M1117732.
- [7] David L Donoho. „Compressed Sensing“. In: *IEEE Transactions on information theory* 52.4 (2006), S. 1289–1306. DOI: 10.1109/TIT.2006.871582.
- [8] Yonina C Eldar und Gitta Kutyniok. *Compressed Sensing: Theory and Applications*. Cambridge university press, 2012. DOI: 10.1017/CB09780511794308.
- [9] William Feller. *An Introduction to Probability Theory and its Applications, Volume 2*. Bd. 2. John Wiley & Sons, 1991.
- [10] Gene H Golub und Charles F Van Loan. *Matrix Computations*. JHU press, 2013.
- [11] Charles R. Harris u. a. „Array programming with NumPy“. In: *Nature* 585.7825 (Sep. 2020), S. 357–362. DOI: 10.1038/s41586-020-2649-2.

-
- [12] Daniel Heldt u. a. „Approximate Computation of Zero-Dimensional Polynomial Ideals“. In: *Journal of Symbolic Computation* 44.11 (2009). In Memoriam Karin Gattermann, S. 1566–1591. ISSN: 0747-7171. DOI: <https://doi.org/10.1016/j.jsc.2008.11.010>. URL: <https://www.sciencedirect.com/science/article/pii/S0747717109000935>.
- [13] Nicholas J Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, 2002. DOI: 10.1137/1.9780898718027.
- [14] David S Johnson und Michael R Garey. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. WH Freeman, 1979.
- [15] Christian Karpfinger und Hellmuth Stachel. *Lineare Algebra*. Springer, 2020. DOI: <https://doi.org/10.1007/978-3-662-61340-5>.
- [16] Leonid G Khachiyan. „Polynomial Algorithms in Linear Programming“. In: *USSR Computational Mathematics and Mathematical Physics* 20.1 (1980), S. 53–72. DOI: [https://doi.org/10.1016/0041-5553\(80\)90061-0](https://doi.org/10.1016/0041-5553(80)90061-0).
- [17] John R Koza. „Genetic Programming as a Means for Programming Computers by Natural Selection“. In: *Statistics and computing* 4.2 (1994), S. 87–112. DOI: 10.1007/BF00175355.
- [18] Jan Kronqvist, Ruth Misener und Calvin Tsay. „P-Split Formulations: A Class of Intermediate Formulations Between Big-M and Convex Hull for Disjunctive Constraints“. In: *arXiv preprint arXiv:2202.05198* (2022). DOI: <https://doi.org/10.48550/arXiv.2202.05198>.
- [19] Stephen Maher u. a. „PySCIPOpt: Mathematical Programming in Python with the SCIP Optimization Suite“. In: *International Congress on Mathematical Software*. Springer. 2016, S. 301–307. DOI: https://doi.org/10.1007/978-3-319-42432-3_37.
- [20] David R Morrison u. a. „Branch-and-Bound Algorithms: A Survey of Recent Advances in Searching, Branching, and Pruning“. In: *Discrete Optimization* 19 (2016), S. 79–102. DOI: <https://doi.org/10.1016/j.disopt.2016.01.005>.
- [21] Balas Kausik Natarajan. „Sparse Approximate Solutions to Linear Systems“. In: *SIAM journal on computing* 24.2 (1995), S. 227–234. DOI: <https://doi.org/10.1137/S0097539792240406>.
- [22] Thanh T Nguyen u. a. „NP-Hardness of l0 Minimization Problems: Revision and Extension to the Non-Negative Setting“. In: *2019 13th International conference on Sampling Theory and Applications (SampTA)*. IEEE. 2019, S. 1–4. DOI: 10.1109/SampTA45681.2019.9030937.

-
- [23] Victor Y Pan. „How Bad are Vandermonde Matrices?“ In: *arXiv preprint arXiv:1504.02118* (2015). DOI: <https://doi.org/10.1137/15M1030170>.
- [24] Salvador Pineda u. a. „Tight Big-Ms for Optimal Transmission Switching“. In: *Electric Power Systems Research* 234 (2024), S. 110620. DOI: <https://doi.org/10.1016/j.epsr.2024.110620>.
- [25] Daniel Plaumann. „Einführung in die Algebraische Geometrie“. In: *Springer Spektrum* 10 (2020). DOI: <https://doi.org/10.1007/978-3-662-61779-3>.
- [26] Python Software Foundation. *Python 3.13.5*. Version 3.13.5. 2025. URL: <https://python.org>.
- [27] Kevin Ryan, Deepak Rajan und Shabbir Ahmed. „Scenario Decomposition for 0-1 Stochastic Programs: Improvements and Asynchronous Implementation“. In: *2016 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE. 2016, S. 722–729. DOI: [10.1109/IPDPSW.2016.119](https://doi.org/10.1109/IPDPSW.2016.119).
- [28] Martin WP Savelsbergh. „Preprocessing and Probing Techniques for Mixed Integer Programming Problems“. In: *ORSA journal on computing* 6.4 (1994), S. 445–454. DOI: <https://doi.org/10.1287/ijoc.6.4.445>.
- [29] Michael Schmidt und Hod Lipson. „Distilling Free-Form Natural Laws from Experimental Data“. In: *science* 324.5923 (2009), S. 81–85. DOI: [10.1126/science.1165893](https://doi.org/10.1126/science.1165893).
- [30] Hanif D Sherali und J Cole Smith. „Improving Discrete Model Representations via Symmetry Considerations“. In: *Management Science* 47.10 (2001), S. 1396–1407. DOI: <https://doi.org/10.1287/mnsc.47.10.1396.10265>.
- [31] Ken Shoemake. „Uniform Random Rotations“. In: *Graphics Gems III (IBM Version)*. Elsevier, 1992, S. 124–132. DOI: <https://doi.org/10.1016/B978-0-08-050755-2.50036-1>.
- [32] Andreas M Tillmann und Marc E Pfetsch. „The Computational Complexity of the Restricted Isometry Property, the Nullspace Property, and Related Concepts in Compressed Sensing“. In: *IEEE Transactions on Information Theory* 60.2 (2013), S. 1248–1259. DOI: <https://doi.org/10.48550/arXiv.1205.2081>.
- [33] Lloyd N Trefethen und David Bau. *Numerical Linear Algebra*. SIAM, 2022. DOI: <https://doi.org/10.1137/1.9780898719574>.
- [34] Silviu-Marian Udrescu und Max Tegmark. „AI Feynman: A Physics-Inspired Method for Symbolic Regression“. In: *Science advances* 6.16 (2020), eaay2631. DOI: <https://doi.org/10.48550/arXiv.1905.11481>.

-
- [35] Ivan Vazler, Kristian Sabo und Rudolf Scitovski. „Weighted Median of the Data in Solving Least Absolute Deviations Problems“. In: *Communications in Statistics-Theory and Methods* 41.8 (2012), S. 1455–1465. DOI: 10.1080/03610926.2010.539750.
- [36] Juan Pablo Vielma. „Mixed Integer Linear Programming Formulation Techniques“. In: *Siam Review* 57.1 (2015), S. 3–57. DOI: <https://doi.org/10.1137/130915303>.
- [37] H Paul Williams. „Integer Programming“. In: *Logic and integer programming*. Springer, 2009, S. 25–70. DOI: <https://doi.org/10.1007/978-0-387-92280-5>.